UNIVERSIDADE DE TRÁS-OS-MONTES E ALTO DOURO

LABORATÓRIO DE APLICAÇÕES WEB E BASES DE DADOS

RELATÓRIO

DO PROJETO

- **António Fernandes – 78207**
- **Francisco Guerra – 78818**
- **José Brás – 78238**
- **João Sousa – 77225**

2024/2025

ÍNDICE

1 Introdução 3

2 FASE 1 3

2.1 Análise dos requisitos do sistema 3

2.2 Diagramas Entidade-Relacionamento(E-R) 4

2.3 Requisitos Funcionais 5

2.4 Diagramas de Casos-de-Uso 9

3 FASE 2 11

3.1 Mapeamento entre o modelo conceptual de dados e o modelo relacional 11

3.2 Implementação do modelo físico da base de dados recorrendo à linguagem sql 12

3.3 Especificação das interfaces com o utilizador para backoffice e o frontoffice recorrendo a mockups 14

4 Fase 3 27

4.1 Apresentação Da Biblioteca 27

4.2 Funcionalidades Importantes 44

5 Considerações Finais 54

1 INTRODUÇÃO

No âmbito da unidade curricular de Laboratório de Aplicações Web e Bases de Dados, foi-nos proposto o desenvolvimento de aplicação *web* que permita suportar a gestão de uma biblioteca.

A Fase 1 do projeto, tem como objetivo a realização de atividades cruciais para o sucesso do desenvolvimento da plataforma, nomeadamente a análise dos requisitos do sistema, especificação do modelo conceptual de dados do sistema, recorrendo a diagramas Entidade-Relacionamento(E-R), procede-se à análise dos requisitos funcionais do sistema com a elaboração do respetivo modelo funcional, representado por diagramas de Casos-de-Uso.

A Fase 2 do projeto, tem como objetivo o desenvolvimento e implementação de uma base de dados completa, abrangendo desde o modelo conceptual até à sua aplicação prática. Nesta fase, será apresentado o mapeamento entre o modelo conceptual de dados e o modelo relacional, seguido da implementação do modelo físico utilizando a linguagem SQL. Além disso, serão especificadas as interfaces para os utilizadores através de *mockups* que ilustram a interação esperada com o sistema.

A Fase 3 consolidou o projeto, dando enfoque à integração do sistema e à implementação das suas funcionalidades completas. Esta etapa incluiu a elaboração de um relatório detalhado sobre a execução do trabalho, documentando os processos e componentes desenvolvidos. Foi assegurada a integridade da base de dados, garantindo o correto funcionamento e manipulação dos dados. Além disso, implementou-se a lógica funcional do sistema, estabelecendo a ligação entre as interfaces de utilizador e a base de dados, tornando a aplicação web funcional e pronta para as fases de testes e validação final.

2 FASE 1

2.1 ANÁLISE DOS REQUISITOS DO SISTEMA

O sistema de gestão de biblioteca deve atender a quatro tipos de utilizadores: utilizadores não autenticados, leitores, bibliotecários e administradores, cada um com necessidades específicas. Utilizadores não autenticados precisam visualizar informações gerais e o catálogo de livros. Leitores devem poder pesquisar, requisitar livros e consultar o histórico. Bibliotecários precisam gerir o catálogo de livros e os empréstimos, além de serem notificados sobre devoluções em atraso. Administradores devem gerir contas e aprovar bibliotecários.

Os requisitos funcionais foram organizados para capturar essas funcionalidades: RF1 a RF2 focam-se nos utilizadores não autenticados, RF3 a RF5 nos leitores, RF6 a RF8 nos bibliotecários e RF9 a RF17 nos administradores. A divisão de responsabilidades foi feita para garantir que o sistema seja eficiente e fácil de usar, atendendo às necessidades de todos os utilizadores.

O diagrama de casos de uso deste projeto facilita a compreensão de como os diferentes tipos de utilizadores interagem com o sistema. Ele organiza e especifica as funções que cada perfil de utilizador pode realizar, desde tarefas simples para utilizadores não autenticados até operações complexas e de gestão realizadas por bibliotecários e administradores. Isso assegura que todas as funcionalidades são documentadas de forma clara, permitindo uma implementação consistente das interações no sistema.

2.2 DIAGRAMAS ENTIDADE-RELACIONAMENTO(E-R)

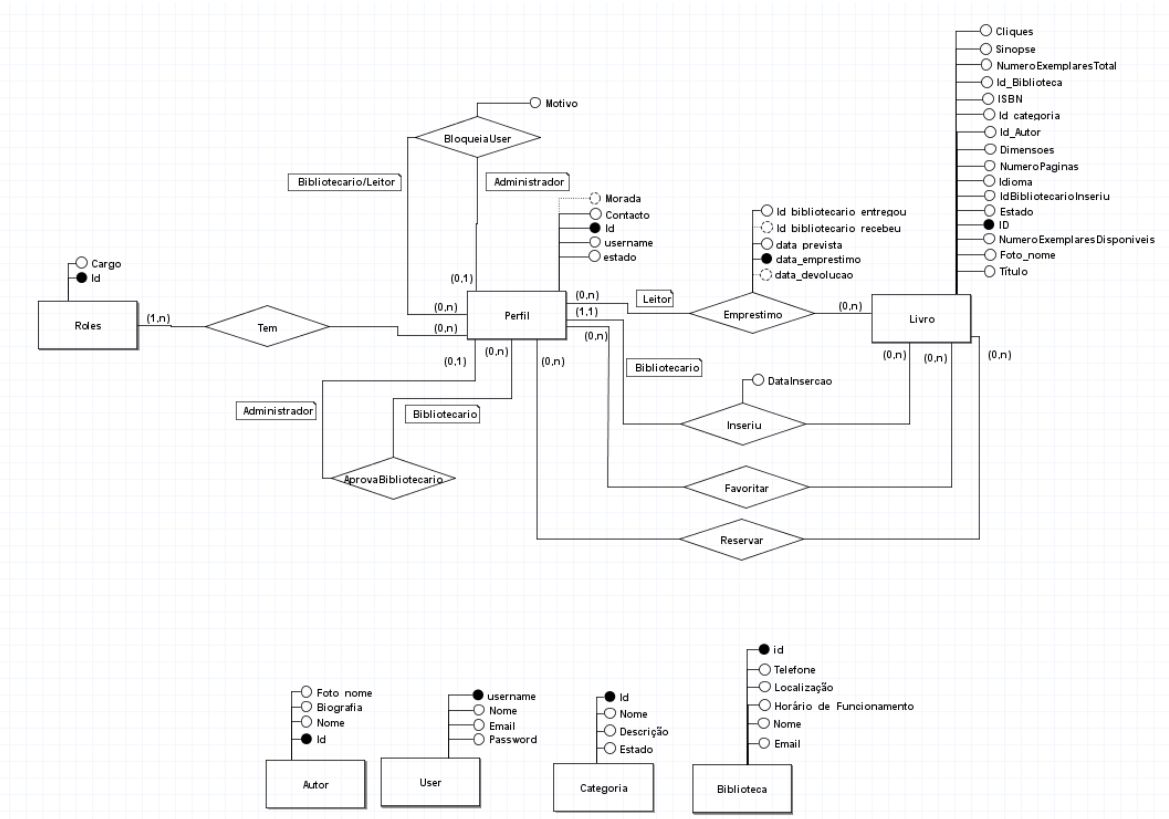


Figura 1 - Diagrama de Entidade-Relacionamento para o sistema de gestão da biblioteca.

Legenda:

1. Entidades:

- **Roles** – Representa os diferentes papéis que um utilizador pode assumir, como ‘Bibliotecário’, ‘Leitor’ e ’Administrador’.
- **User** – Representa os utilizadores do sistema, com atributos como ‘username’, ‘email’, ‘password’ e ‘estado’.
- **Perfil** - Representa o perfil de um utilizador, associado aos dados como 'morada', ‘contacto’ e ‘id’.
- **Emprestimo** - Representa a transação de empréstimo de livros, com atributos como 'data prevista', 'data empréstimo' e 'data devolução'.
- **Livro** – Representa os livros disponíveis na biblioteca, com atributos como 'nome', 'autor', 'preço', ‘editora’ e ‘ISBN’.
- **Categoria** – Classifica os livros em diferentes categorias com base no ‘nome’ e ‘descrição’.
- **Biblioteca** – Representa informações da biblioteca, como ‘nome’, ‘telefone’, 'localização', 'horário de funcionamento', e 'email'.
- **Autor** – Representa a transação de empréstimo de livros, com informações como ‘data_prevista’, ‘data_emprestimo’ e ‘data_devolucao’.

2. Relacionamentos
    - **Livro - Emprestimo (1..N):** Um livro pode ser emprestado múltiplas vezes.
    - **Livro - Categoria (1..N):** Um livro pertence a uma ou mais categorias.
    - **Livro - Biblioteca (N..1):** Múltiplos livros podem pertencer a uma única biblioteca.
    - **Livro – Autor (1..1):** Cada livro é escrito por um autor.
    - **AprovaBibliotecario (0..N):** O administrador pode aprovar bibliotecários.
    - **BloqueiaUser:** Um utilizador pode ser bloqueado, onde se deve indicar o motivo do bloqueio.

2.3 Requisitos Funcionais

RF1 O sistema deverá permitir que utilizadores não autenticados consultem informações gerais sobre a biblioteca, incluindo nome, localização, email, telefone e horário de funcionamento.

Descrição: Utilizadores que não efetuaram login no sistema poderão visualizar informações básicas da biblioteca, incluindo nome, localização, email, telefone e horário de funcionamento.

RF2 O sistema deverá permitir que utilizadores não autenticados visualizem as listagens de livros disponíveis, apresentando detalhes como título, autor, género, preço, ISBN e número de exemplares.

Descrição: Utilizadores não autenticados poderão aceder a uma lista dos livros disponíveis na biblioteca, com detalhes como título, autor, género, preço, ISBN e número de exemplares em stock.

RF3 O sistema deverá permitir que leitores pesquisem livros por título, autor, género ou outros critérios.

Descrição: O sistema permitirá que os leitores realizem pesquisas no catálogo de livros, utilizando critérios como título, autor, género, ou outros filtros específicos, para facilitar a pesquisa de obras.

RF4 O sistema deverá possibilitar aos leitores efetuar a requisição de livros, estabelecendo uma data limite para devolução (máximo de quinze dias).

Descrição: Leitores autenticados poderão requisitar livros através do sistema, selecionando um livro disponível e definindo uma data de devolução, com um limite máximo de quinze dias.

RF5 O sistema deverá fornecer acesso ao histórico de livros requisitados pelo leitor.

Descrição: Leitores poderão consultar o histórico completo de livros que requisitaram anteriormente, visualizando detalhes como datas de requisição e devolução.

RF6 O sistema deverá permitir que os bibliotecários possam adicionar, editar ou remover livros do catálogo da biblioteca.

Descrição: Bibliotecários autenticados terão acesso a funcionalidades que lhes permitem gerir o catálogo de livros, podendo inserir novos livros, alterar informações existentes ou remover livros do sistema.

RF7 O sistema deverá permitir que bibliotecários visualizem os empréstimos em curso, incluindo informações sobre quem requisitou cada livro e a data prevista de devolução.

Descrição: Bibliotecários poderão ver uma lista de todos os empréstimos ativos, com informações sobre quem requisitou cada livro, a data da requisição e a data prevista para devolução.

RF8 O sistema deverá permitir, ao efetuar login, que os bibliotecários sejam notificados sobre leitores com entregas em atraso.

Descrição: Ao efetuar login, os bibliotecários serão informados automaticamente sobre quaisquer leitores que tenham ultrapassado o prazo de devolução dos livros requisitados, permitindo uma melhor gestão de empréstimos.

RF9 O sistema deverá permitir que administradores criem novos utilizadores com permissões de administrador.

Descrição: Administradores autenticados poderão adicionar novos utilizadores ao sistema, atribuindo permissões de administrador para que possam realizar tarefas administrativas, como gestão de utilizadores e do catálogo.

RF10 O sistema deverá permitir que os administradores possam visualizar e atualizar informações de perfil de leitores e bibliotecários, bem como ativar ou bloquear contas.

Descrição: Administradores terão acesso às informações de perfil de leitores e bibliotecários, podendo atualizá-las, além de terem a capacidade de ativar ou bloquear contas de utilizadores conforme necessário.

RF11 O sistema deverá manter um registo do administrador que aprovou cada pedido de registo de bibliotecário.

Descrição: Sempre que um novo bibliotecário for aprovado, o sistema registará automaticamente o administrador responsável pela aprovação, mantendo um histórico para fins de auditoria e controlo.

RF12 O sistema deverá permitir que os leitores possam registar-se através do portal.

Descrição: Leitores poderão criar uma conta na plataforma preenchendo um formulário de registo online, inserindo informações como nome, email e senha, para obter acesso às funcionalidades disponíveis para utilizadores registados.

RF13 O sistema deverá permitir que, após a inscrição, o leitor receba um email de confirmação para validar o registo. Só após esta validação poderá efetuar login.

Descrição: Após o processo de registo, o leitor receberá um email com um link de confirmação. Somente após clicar nesse link o seu registo será validado e ele poderá aceder ao sistema efetuando login.

RF14 O sistema deverá permitir que os bibliotecários possam registar-se através do portal.

Descrição: Após o processo de registo, o leitor receberá um email com um link de confirmação. Somente após clicar nesse link o seu registo será validado e ele poderá aceder ao sistema efetuando login.

RF15 O sistema deverá exigir que o registo seja aprovado por um administrador. Apenas após essa aprovação, o bibliotecário poderá utilizar as funcionalidades correspondentes.

Descrição: Após o registo inicial, o sistema aguardará a aprovação de um administrador. Só depois da aprovação, o bibliotecário terá acesso às funcionalidades específicas destinadas ao seu perfil.

RF16 O sistema deverá permitir que os administradores possam bloquear contas de leitores ou bibliotecários, indicando o motivo.

Descrição: Administradores poderão bloquear contas de leitores ou bibliotecários diretamente no sistema, devendo fornecer uma justificação clara e objetiva para o bloqueio, que será registada.

RF17 O sistema deverá permitir que os utilizadores bloqueados sejam ser informados da razão ao tentarem aceder ao sistema.

Descrição: Quando um utilizador bloqueado tentar fazer login, o sistema exibirá uma mensagem informando o bloqueio, juntamente com a razão fornecida pelo administrador, para que o utilizador compreenda o motivo da restrição de acesso.

2.4 DIAGRAMAS DE CASOS-DE-USO

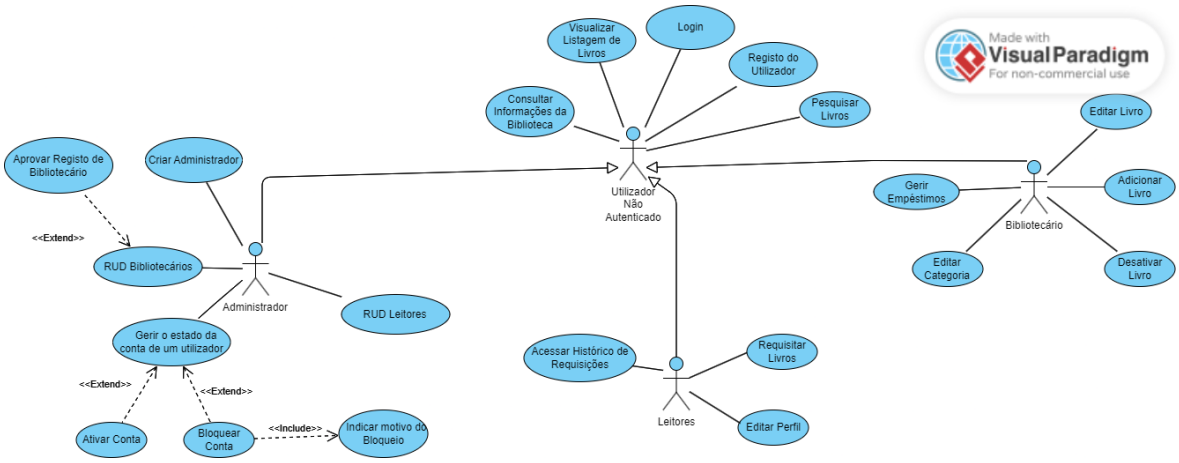


Figura 2 – Diagramas de Casos-de-Uso para o sistema de gestão da biblioteca.

Legenda:

1. Atores:
   - **Utilizador Não Autenticado -** Representa os utilizadores que ainda não se registaram ou não fizeram login no sistema.
   - **Leitores -** Utilizadores com o papel de leitores, que podem requisitar e pesquisar livros, além de aceder o histórico de requisições.
   - **Bibliotecário -** Representa o papel de um utilizador que tem a capacidade de gerir os livros (adicionar, remover, editar) e gerir empréstimos.
   - **Administrador -** Representa um utilizador com permissões avançadas para gerir bibliotecários, leitores e contas de utilizadores.
2. Casos de Uso:
   - <u>Para o Utilizador Não Autenticado:</u>
     - **Login**: Permite que o utilizador faça o login no sistema.
     - **Registo de Utilizador**: Permite que um utilizador não registado, se registe no sistema.
     - **Visualizar Listagem de Livros**: Disponível para utilizadores não autenticados visualizarem os livros disponíveis.
     - **Consultar Informações da Biblioteca**: Fornece informações gerais da biblioteca.
   - <u>Para Leitores:</u>
     - **Requisitar Livros:** Permite que leitores requisitem livros.

- **Pesquisar Livros:** Permite procurar livros no sistema.
- **Aceder ao Histórico de Requisições:** Permite que leitores consultem o histórico de livros requisitados.

- Para Bibliotecários:
  - **Gerir Empréstimos:** Permite aos bibliotecários gerir o processo de empréstimos de livros.
  - **Editar, Adicionar, Remover Livro:** Estas funcionalidades permitem ao bibliotecário gerir o inventário de livros do sistema.
- Para Administradores:
  - **Criar Administrador:** Permite que o administrador crie outros administradores.
  - **Aprovar o Registo de Bibliotecário:** Permite que o administrador aprove novos bibliotecários no sistema.
  - **RUD Bibliotecário:** Funcionalidade para o administrador gerir (ler, atualizar, excluir) bibliotecários.
  - **RUD Leitor:** Funcionalidade para o administrador gerir (ler, atualizar, excluir) leitores.
  - **Gerir o estado da conta de um utilizador:** O administrador pode bloquear ou ativar a conta de alguém.
    - Bloquear Conta: Permite bloquear uma conta, dizendo qual o motivo de bloqueio.
    - Ativar Conta: Permite ativar uma conta bloqueada ou inativa.

3 FASE 2

3.1 MAPEAMENTO ENTRE O MODELO CONCEPTUAL DE DADOS E O MODELO RELACIONAL

Entidades

- Roles (ID, Cargo)
- User (username, Nome, Email, Password)
- Perfil (ID, ~~username~~, Contacto, Morada, Estado)
- Livro (ID, ~~Id_biblioteca~~, ~~id_categoria~~, ~~id_autor~~, ISBN, Dimensoes, NumeroPaginas,Idioma,IdBibliotecarioInseriu,Estado,NumeroExemplaresDisponiveis, NumeroExemplaresTotal, Titulo, Foto_nome)
- Categoria (ID, Nome, Descricao, Estado)
- Autor (ID, Nome, Biografia, Foto_nome)
- Biblioteca (ID, Telefone, Localizacao, Horario_De_Funcionamento, Nome, Email)

Relacionamentos

- Tem (~~IdRoles~~, ~~ID_PerfilUser~~)

  IdRoles referencia Roles

  Id_PerfilUser referencia Perfil

  BloqueiaUser (~~ID_PerfilUser,~~ ~~ID_PerfilAdmin~~ , Motivo)

  ID_PerfilUser, ID_PerfilAdmin referencia Perfil
- AprovaBibliotecario (~~ID_PerfilBibliotecario~~, ~~ID_PerfilAdmin~~)

  ID_PerfilBibliotecario, ID_PerfilAdmin referencia Perfil
- Emprestimo (~~ID_PerfilLeitor~~, ~~ID_Livro~~, data_emprestimo, ~~Id_bibliotecario_entregou~~, ~~Id_bibliotecario_recebeu~~, data_prevista, data_devolucao)

  ID_Livro referencia Livro

  ID_PerfilLeitor, Id_bibliotecario_entregou, Id_bibliotecario_recebeu referencia Perfil
- Inseriu (~~ID_Livro~~), ~~ID_PerfilBibliotecario~~, DataInsercao)

  ID_Livro referencia Livro

  ID_PerfilBibliotecario referencia Perfil
- Favoritar (~~ID_Livro~~, ~~ID_PerfilLeitor~~)

  ID_Livro referencia Livro

  ID_PerfilLeitor referencia Perfil
- Reservar (~~ID_Livro~~, ~~ID_PerfilLeitor~~)

ID_Livro referencia Livro

ID_PerfilLeitor referencia Perfil

3.2 IMPLEMENTAÇÃO DO MODELO FÍSICO DA BASE DE DADOS RECORRENDO À LINGUAGEM SQL

```sql
CREATE DATABASE db;
GO

USE db;
GO

-- Tabela de Roles
CREATE TABLE Roles (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Cargo VARCHAR(100) NOT NULL CHECK (Cargo IN ('Admin', 'Bibliotecario', 'Leitor'))
);

-- Tabela de Users
CREATE TABLE Utilizador (
    username VARCHAR(50) PRIMARY KEY NOT NULL,
    Nome VARCHAR(100) NOT NULL CHECK (Nome <> ''),
    Email VARCHAR(100) UNIQUE NOT NULL,
    Password VARCHAR(20) NOT NULL,
);

-- Tabela de Perfil
CREATE TABLE Perfil (
    Id INT PRIMARY KEY IDENTITY(1,1) NOT NULL,
    username VARCHAR(50),
    Contacto VARCHAR(13) CHECK (Contacto LIKE '+351[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]' OR
Contacto IS NULL),
    Morada VARCHAR(255),
    Estado VARCHAR(50) CHECK (Estado IN ('PorAtivar', 'Ativo', 'Bloqueado')),
    FOREIGN KEY (username) REFERENCES Utilizador(username)
);

-- Tabela de Autor
CREATE TABLE Autor (
    Id INT PRIMARY KEY IDENTITY(1,1) NOT NULL,
    Nome VARCHAR(100) NOT NULL CHECK (Nome <> ''),
    Biografia VARCHAR(100) NOT NULL CHECK (Biografia <> ''),
    Foto_nome VARCHAR(255)
);

-- Tabela de Categoria
CREATE TABLE Categoria (
    Id INT PRIMARY KEY IDENTITY(1,1) NOT NULL,
    Nome VARCHAR(20) NOT NULL CHECK (Nome <> ''),
    Descricao VARCHAR(100) NOT NULL CHECK (Descricao <> ''),
    Estado BIT NOT NULL --0 -> desativada / 1 -> ativada
);

-- Tabela de Biblioteca
CREATE TABLE Biblioteca (
    Id INT PRIMARY KEY IDENTITY(1,1) NOT NULL,
    Telefone VARCHAR(13) CHECK (
        Telefone LIKE '+351[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'
        OR Telefone IS NULL
    ),
    Localizacao VARCHAR(255),
    Horario_De_Funcionamento VARCHAR(100),
    Nome VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE
);
```

Figura 3 - Modelo Físico de Base de Dados em Linguagem SQL.

```sql
-- Tabela de Livro
CREATE TABLE Livro (
    ID INT PRIMARY KEY IDENTITY(1,1) NOT NULL,
    Id_biblioteca INT NOT NULL,
    id_categoria INT NOT NULL,
    id_autor INT NOT NULL,
    ISBN VARCHAR(20) NOT NULL CHECK (
        (ISBN LIKE '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]X' OR  -- ISBN-10 com 'X'
         ISBN LIKE '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]' OR  -- ISBN-10 numérico
         ISBN LIKE '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]')  -- ISBN-13
    ),
    Dimensoes VARCHAR(50) NOT NULL CHECK (
        Dimensoes LIKE '[0-9]% x [0-9]% x [0-9]% mm'
    ),
    NumeroPaginas INT CHECK (NumeroPaginas > 0),
    Idioma VARCHAR(50) NOT NULL CHECK (Idioma <> ''),
    IdBibliotecarioInseriu INT NOT NULL,
    Estado VARCHAR(50) CHECK (Estado IN ('Disponível', 'Indisponivel')),
    NumeroExemplaresDisponiveis INT CHECK (NumeroExemplaresDisponiveis > 0),
    NumeroExemplaresTotal INT CHECK (NumeroExemplaresTotal > 0),
    Titulo VARCHAR(255) NOT NULL,
    Foto_nome VARCHAR(255) NOT NULL,
    Cliques INT CHECK (Cliques >= 0) NOT NULL,
    Sinopse VARCHAR(500) NOT NULL,
    FOREIGN KEY (Id_biblioteca) REFERENCES Biblioteca(Id),
    FOREIGN KEY (id_categoria) REFERENCES Categoria(Id),
    FOREIGN KEY (id_autor) REFERENCES Autor(Id),
    FOREIGN KEY (IdBibliotecarioInseriu) REFERENCES Perfil(Id)
);

-- Tabela de Tem (Relacionamento entre Roles e Perfil)
CREATE TABLE Tem (
    IdRoles INT,
    ID_PerfilUser INT,
    PRIMARY KEY (IdRoles, ID_PerfilUser),
    FOREIGN KEY (IdRoles) REFERENCES Roles(Id),
    FOREIGN KEY (ID_PerfilUser) REFERENCES Perfil(Id)
);

-- Tabela de BloqueiaUser (Relacionamento entre Perfil e Perfil)
CREATE TABLE BloqueiaUser (
    ID_PerfilUser INT,
    ID_PerfilAdmin INT,
    Motivo VARCHAR(100) CHECK (Motivo <> ''),
    PRIMARY KEY (ID_PerfilUser, ID_PerfilAdmin),
    FOREIGN KEY (ID_PerfilUser) REFERENCES Perfil(Id),
    FOREIGN KEY (ID_PerfilAdmin) REFERENCES Perfil(Id)
);

-- Tabela de AprovaBibliotecario (Relacionamento entre Perfil e Perfil)
CREATE TABLE AprovaBibliotecario (
    ID_PerfilBibliotecario INT,
    ID_PerfilAdmin INT,
    PRIMARY KEY (ID_PerfilBibliotecario, ID_PerfilAdmin),
    FOREIGN KEY (ID_PerfilBibliotecario) REFERENCES Perfil(Id),
    FOREIGN KEY (ID_PerfilAdmin) REFERENCES Perfil(Id)
);

-- Tabela de Emprestimo
CREATE TABLE Emprestimo (
    ID_PerfilLeitor INT NOT NULL,
    ID_Livro INT NOT NULL,
    data_emprestimo DATETIME NOT NULL DEFAULT GETDATE(),
    Id_bibliotecario_entregou INT NOT NULL,
    Id_bibliotecario_recebeu INT,
    data_prevista DATETIME NOT NULL DEFAULT DATEADD(DAY, 5, GETDATE()),
    data_devolucao DATETIME,
    PRIMARY KEY (ID_PerfilLeitor, ID_Livro, data_emprestimo),
    FOREIGN KEY (ID_PerfilLeitor) REFERENCES Perfil(Id),
    FOREIGN KEY (ID_Livro) REFERENCES Livro(ID),
    FOREIGN KEY (Id_bibliotecario_entregou) REFERENCES Perfil(Id),
    FOREIGN KEY (Id_bibliotecario_recebeu) REFERENCES Perfil(Id),
    CHECK (data_devolucao IS NULL OR data_devolucao > data_prevista)
);

-- Tabela de Inseriu (Relacionamento entre Livro e Perfil)
CREATE TABLE Inseriu (
    ID_Livro INT NOT NULL,
    ID_PerfilBibliotecario INT NOT NULL,
    DataInsercao DATETIME NOT NULL DEFAULT GETDATE(),
    PRIMARY KEY (ID_Livro),
    FOREIGN KEY (ID_Livro) REFERENCES Livro(ID),
    FOREIGN KEY (ID_PerfilBibliotecario) REFERENCES Perfil(Id)
);

-- Tabela de Favoritar (Relacionamento entre Livro e Perfil)
CREATE TABLE Favoritar (
    ID_Livro INT NOT NULL,
    ID_PerfilLeitor INT NOT NULL,
    PRIMARY KEY (ID_Livro, ID_PerfilLeitor),
    FOREIGN KEY (ID_Livro) REFERENCES Livro(ID),
    FOREIGN KEY (ID_PerfilLeitor) REFERENCES Perfil(Id)
);

-- Tabela de Reservar (Relacionamento entre Livro e Perfil)
CREATE TABLE Reservar (
    ID_Livro INT NOT NULL,
    ID_PerfilLeitor INT NOT NULL,
    PRIMARY KEY (ID_Livro, ID_PerfilLeitor),
    FOREIGN KEY (ID_Livro) REFERENCES Livro(ID),
    FOREIGN KEY (ID_PerfilLeitor) REFERENCES Perfil(Id)
```

Figura 4 - Modelo Físico de Base de Dados em Linguagem SQL.

3.3 ESPECIFICAÇÃO DAS INTERFACES COM O UTILIZADOR PARA BACKOFFICE E O FRONTOFFICE RECORRENDO A MOCKUPS



Figura 5 – Página Principal (Utilizador não Autenticado).



Figura 6 – Página Principal (Utilizador Autenticado).



Figura 7 – Página Registo.



Figura 8 – Página Login.



Figura 9 – Página Recuperar Palavra-Passe.

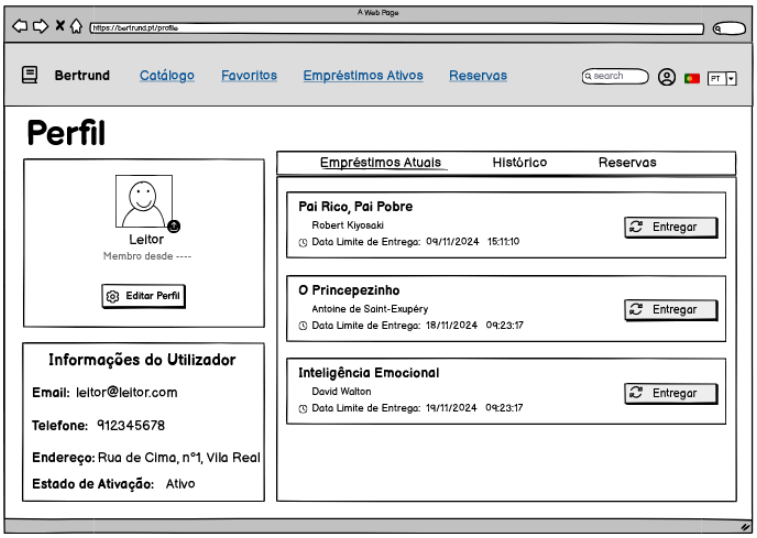


Figura 10 – Página Perfil (Empréstimos Atuais).

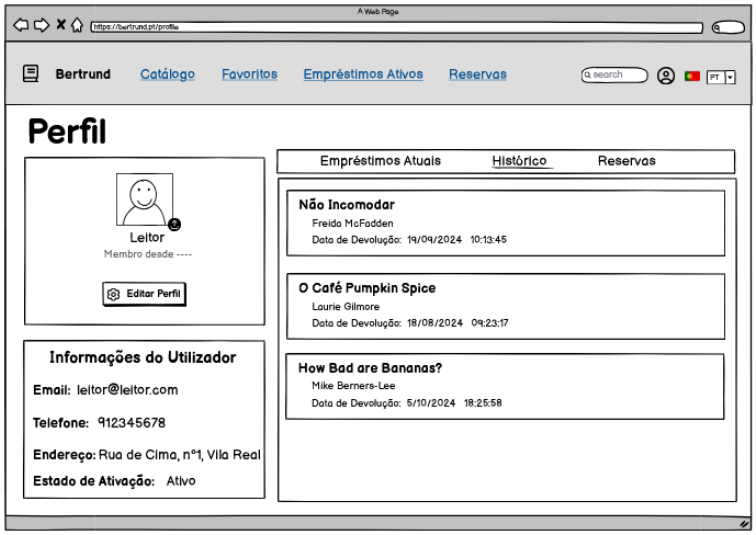


Figura 11 - Perfil (Histórico).



Figura 12 – Página Perfil (Reservas).

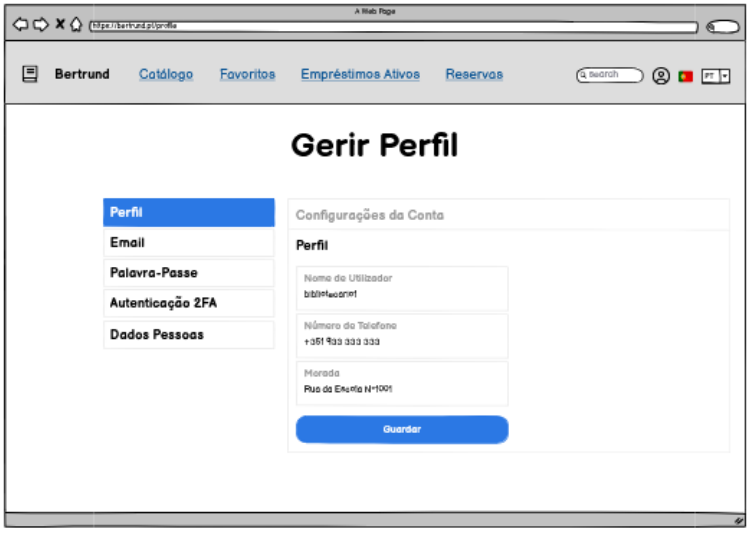


Figura 13 – Página Editar Perfil (Perfil).

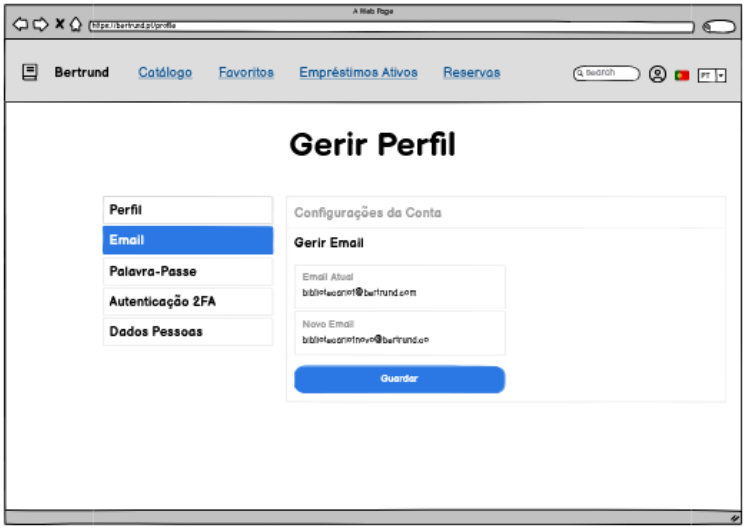


Figura 14 – Página Editar Perfil (Email).

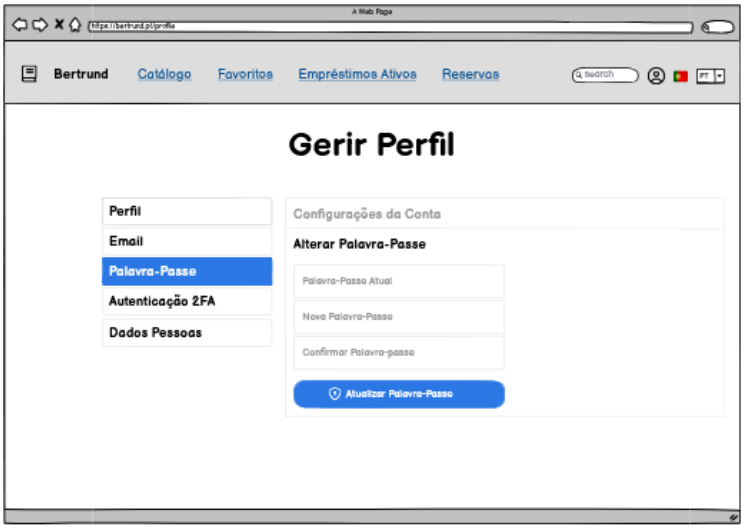


Figura 15 – Página Editar Perfil (Palavra-Passe).



Figura 16 - Página Editar Perfil (Autenticação 2FA)).

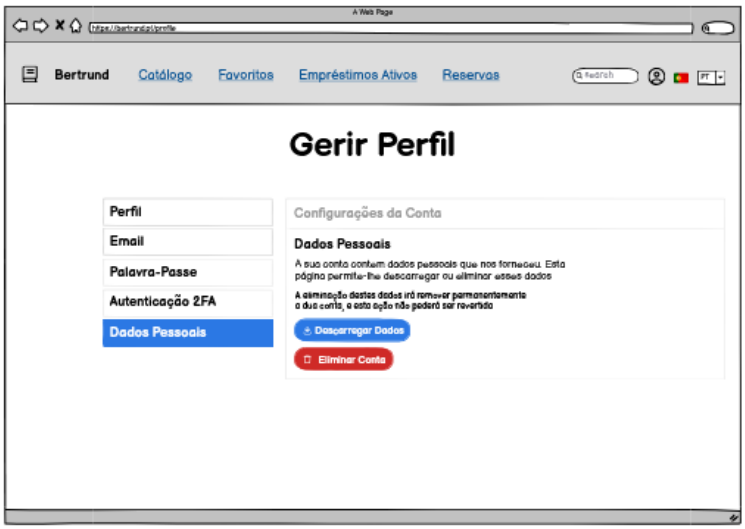


Figura 17 - Página Editar Perfil (Dados Pessoais).



Figura 18 - Página Sobre Nós.

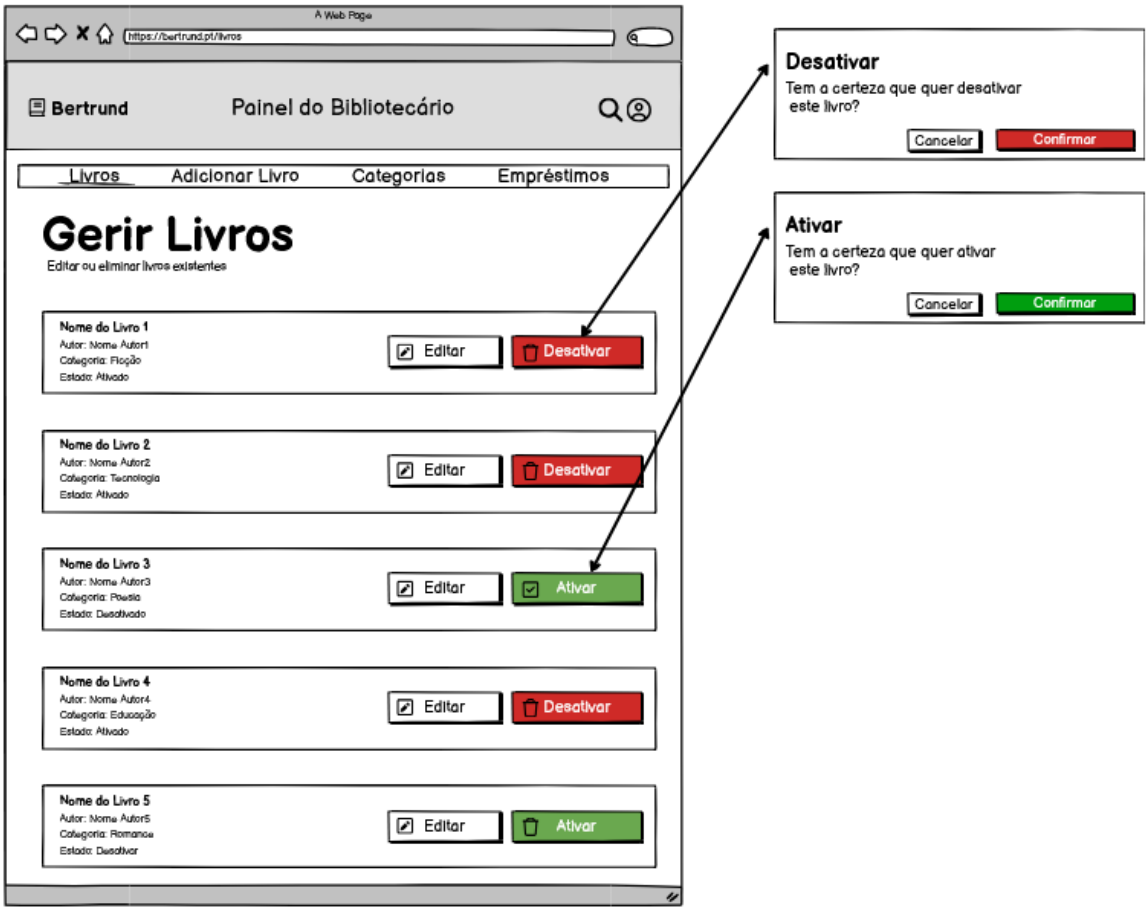


Figura 19 - Página Gerir Livros.

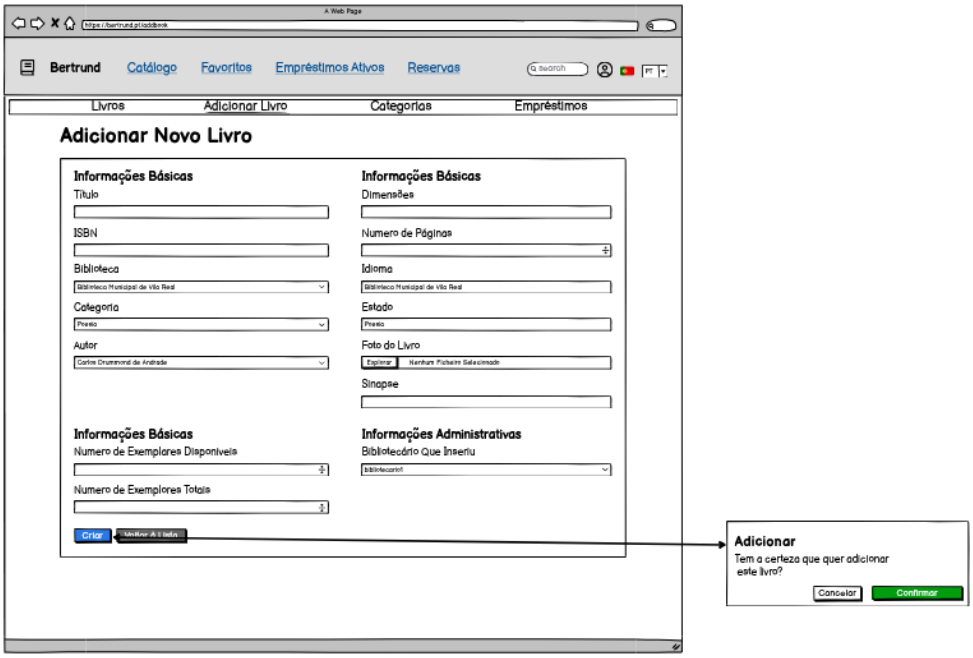


Figura 20 - Página Adicionar Novo Livro.

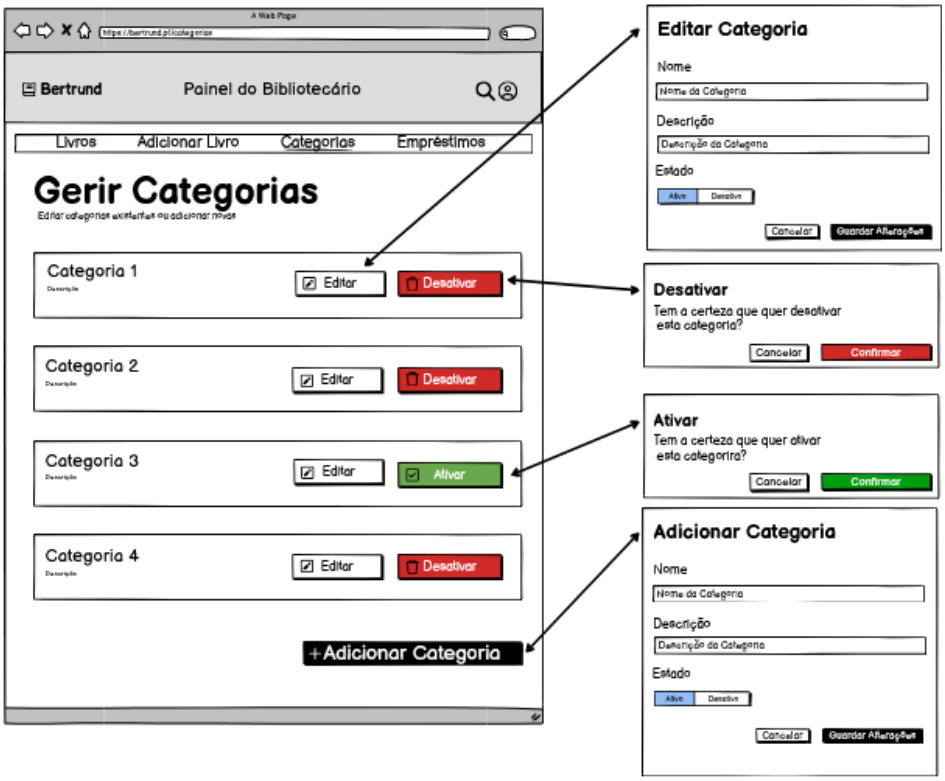


Figura 21 - Página Gerir Categorias.

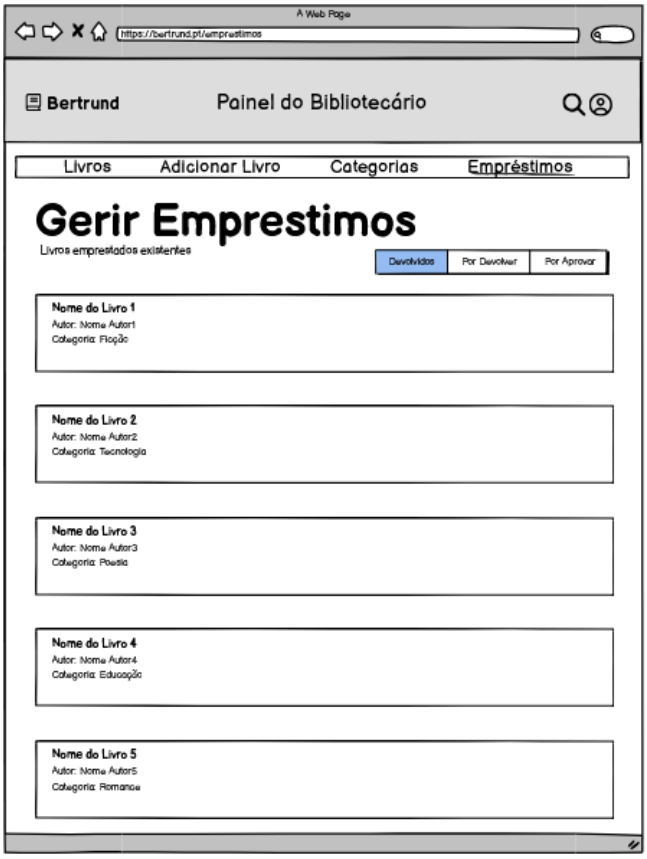


Figura 22 - Página Gerir Empréstimos (Devolvidos).



Figura 23 - Página Gerir Empréstimos (Por Devolver).

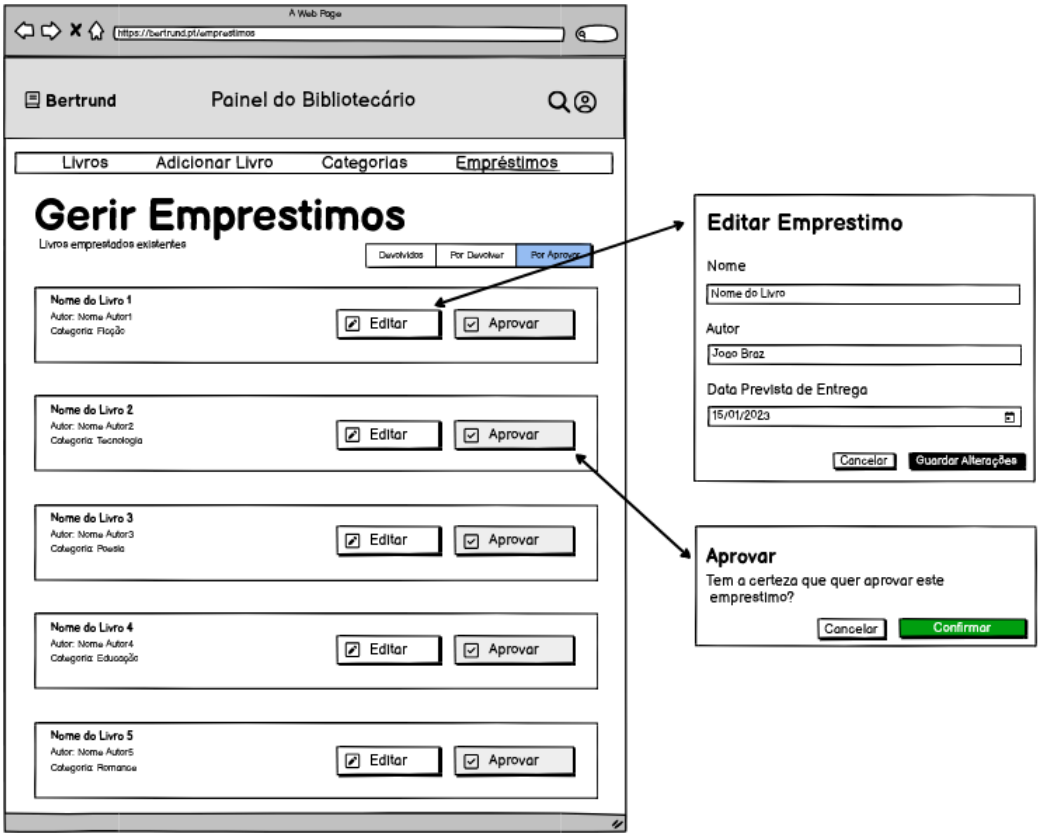


Figura 24 - Página Gerir Empréstimos (Por Aprovar).

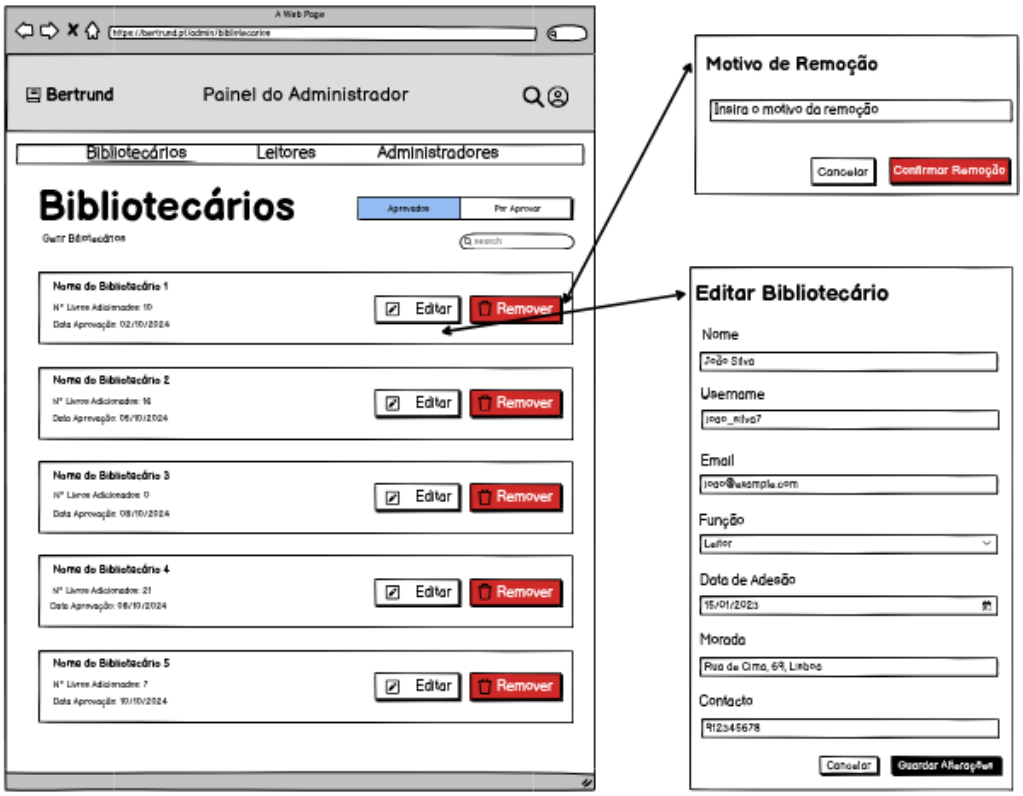


Figura 25 - Página Gerir Bibliotecários (Aprovados).

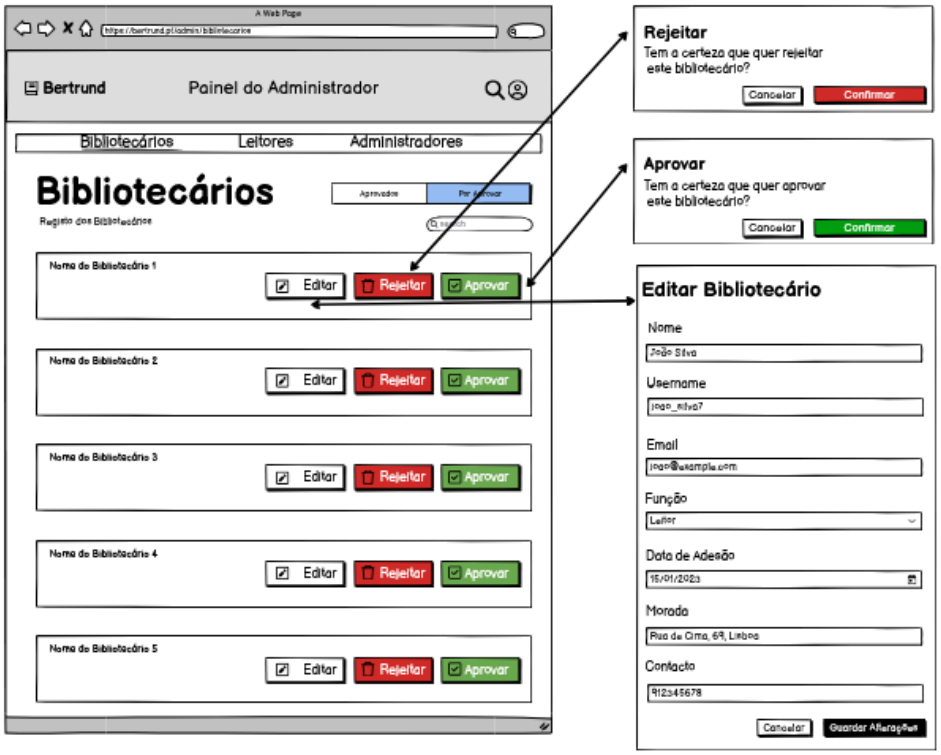


Figura 26 - Página Gerir Bibliotecários (Por Aprovar).



Figura 27 - Página Gerir Leitores.



Figura 28 - Página Gerir Administradores.

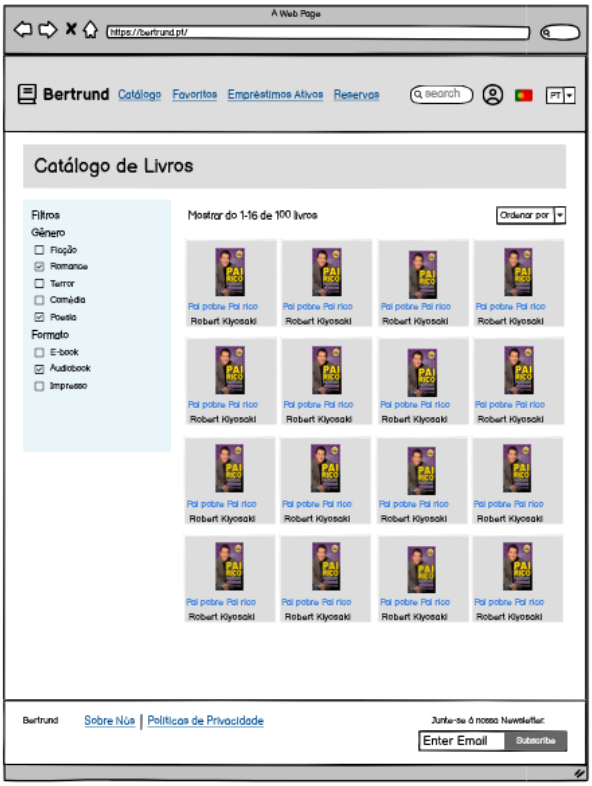


Figura 29 - Página Catálogo.

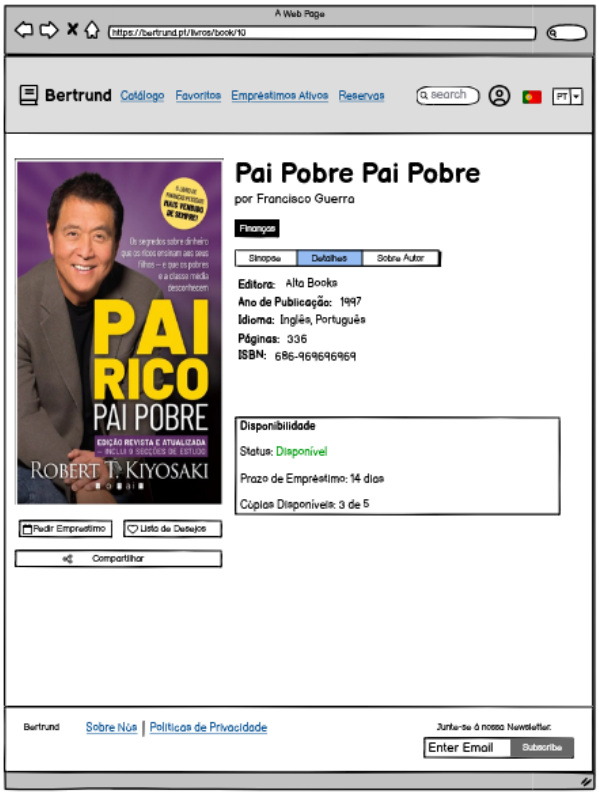


Figura 30 - Página Genérica Livro Disponível.

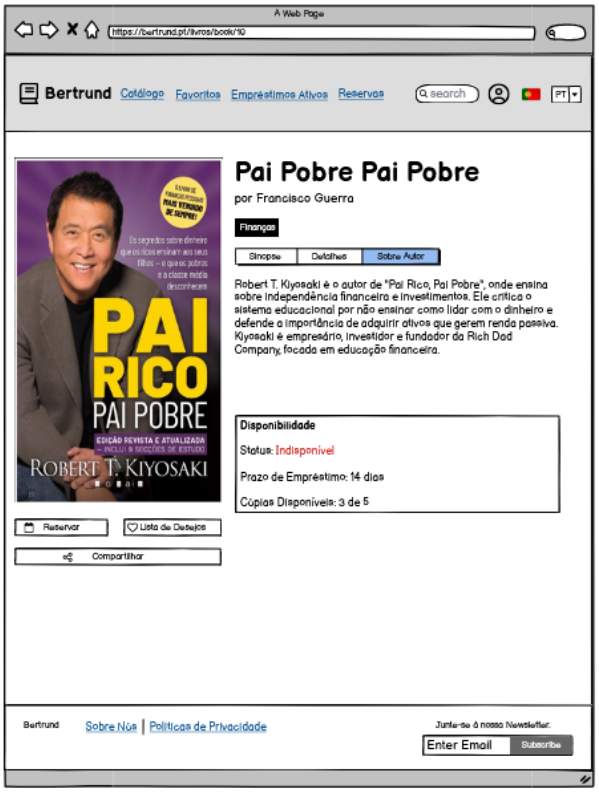


Figura 31 - Página Genérica Livro Indisponível.

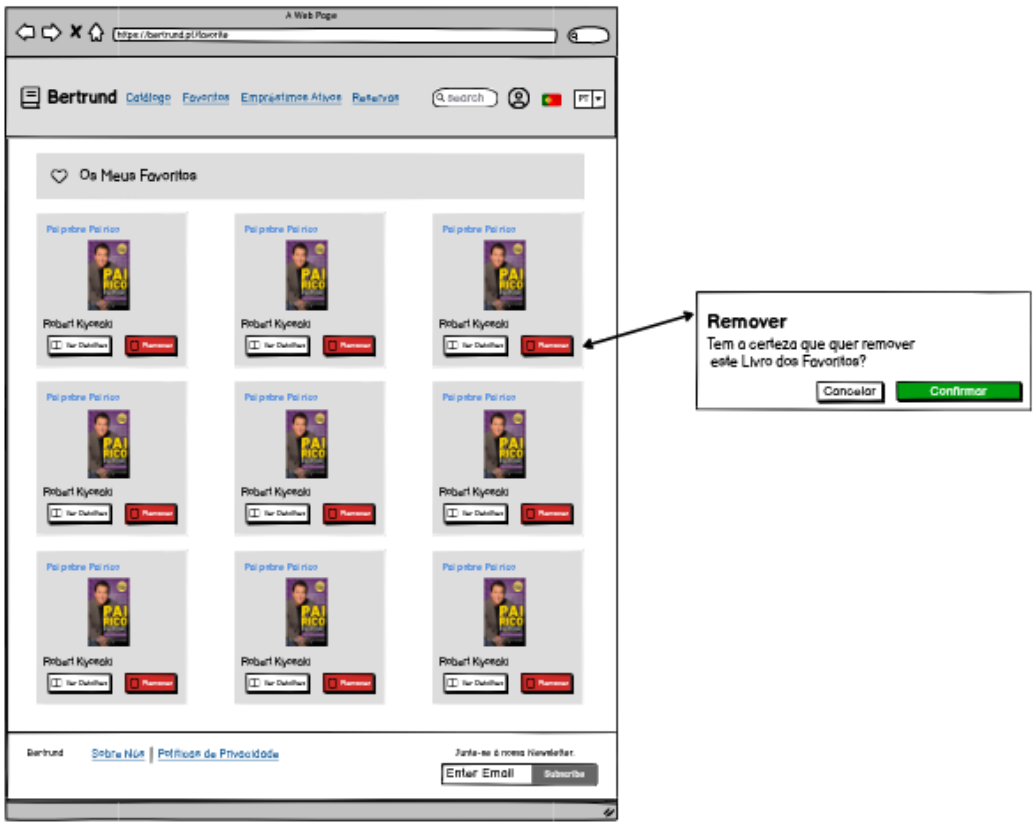


Figura 32 - Página Favoritos.

4 FASE 3

4.1 APRESENTAÇÃO DA BIBLIOTECA

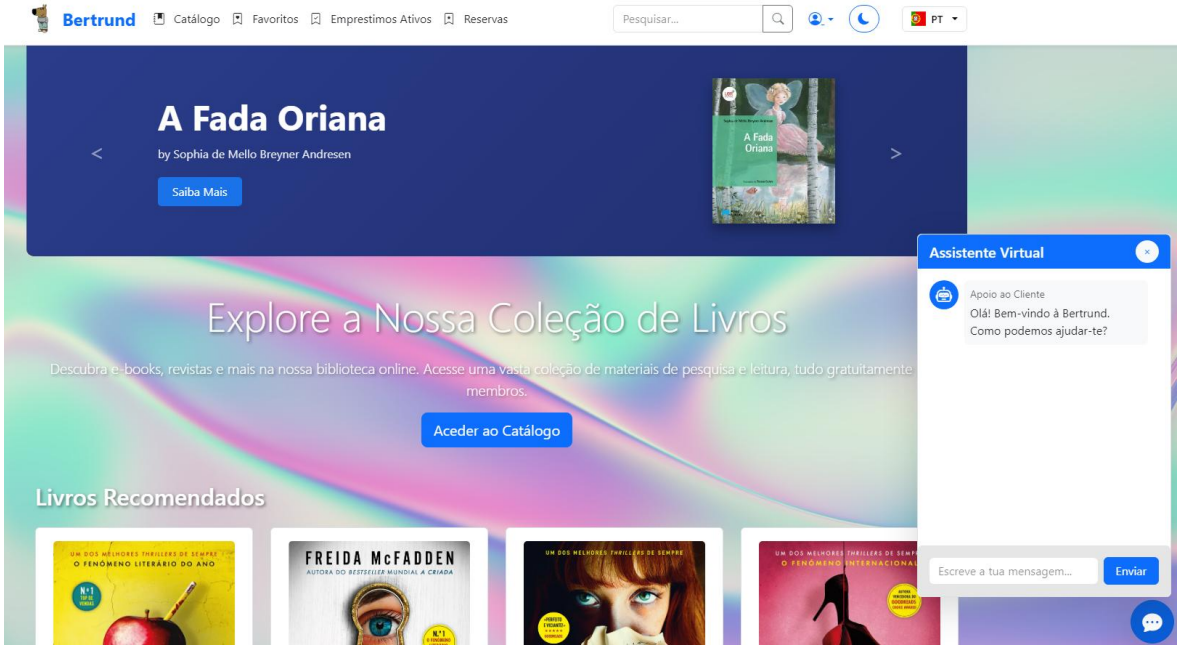


Figura 33 - Página inicial da biblioteca virtual 'Bertrund'.

Página inicial da biblioteca, apresenta a secção "Explore a Nossa Coleção de Livros", incentivando o acesso ao catálogo. Inclui livros recomendados e a presença de um assistente virtual no canto inferior direito, permitindo interações em tempo real para suporte ao cliente.

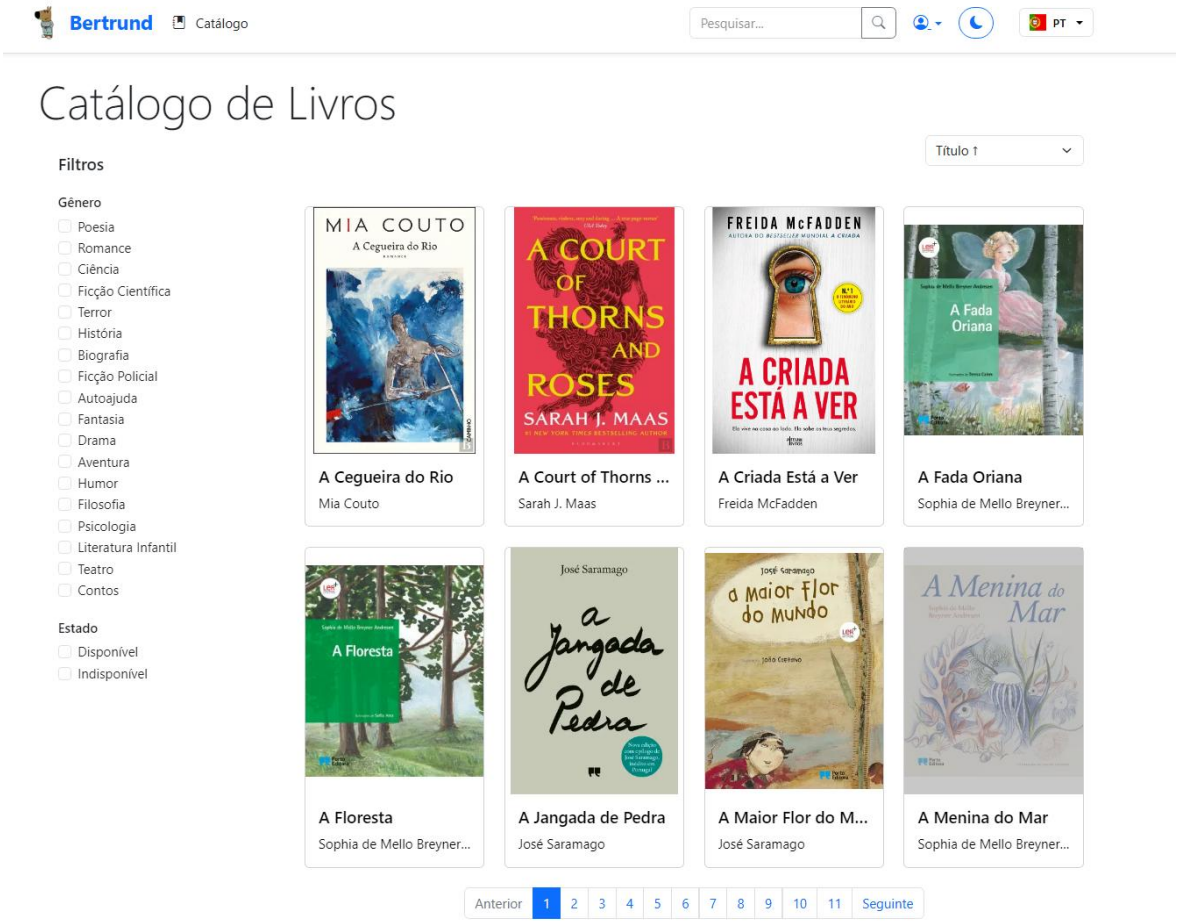


Figura 34 – Catálogo da Biblioteca.

Catálogo da biblioteca "Bertrund" com funcionalidades de filtros por género e disponibilidade, exibição de capas e títulos, e navegação paginada.

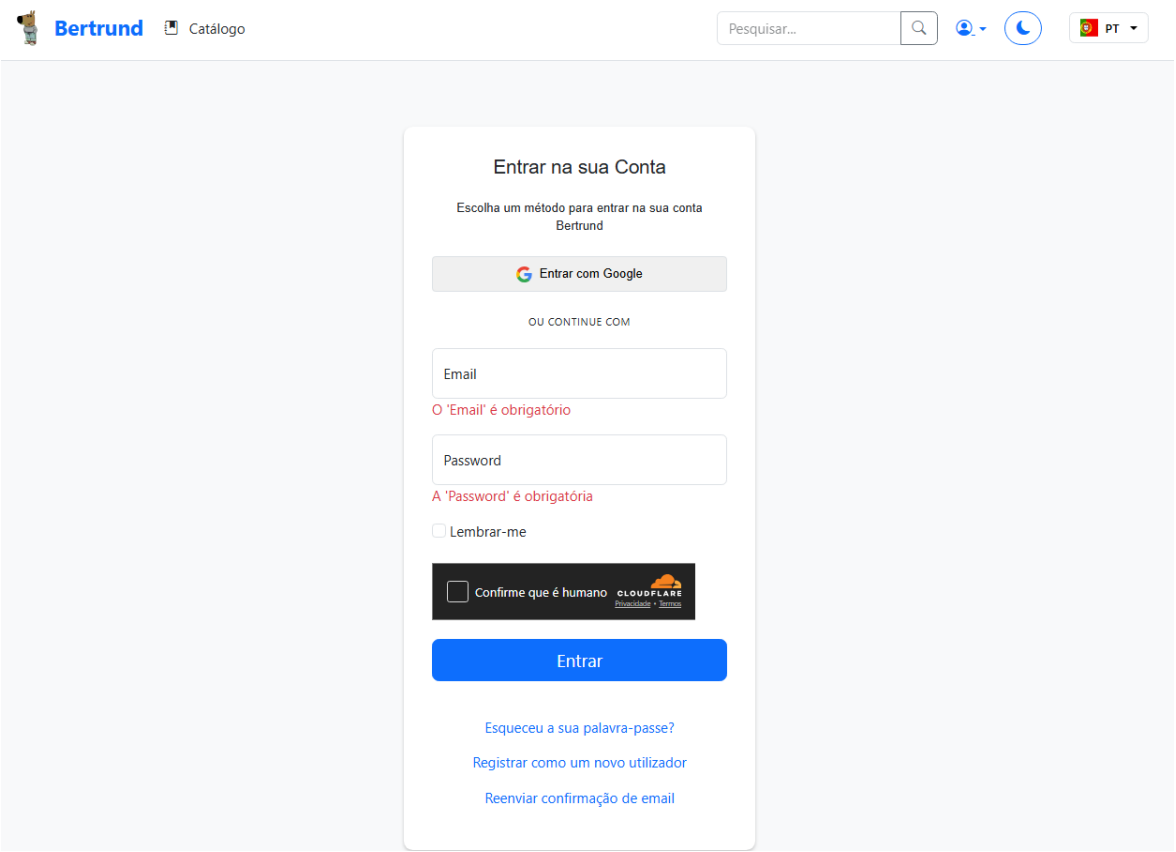


Figura 35 – Página de login da biblioteca.

Página de login da biblioteca, com opções para autenticação através de Google ou inserção manual de email e palavra-passe. Exibe mensagens de erro caso os campos obrigatórios não sejam preenchidos. Inclui funcionalidades como "Lembrar-me", verificação CAPTCHA e links para recuperação de palavra-passe, registo de novos utilizadores e reenvio de confirmação de email.

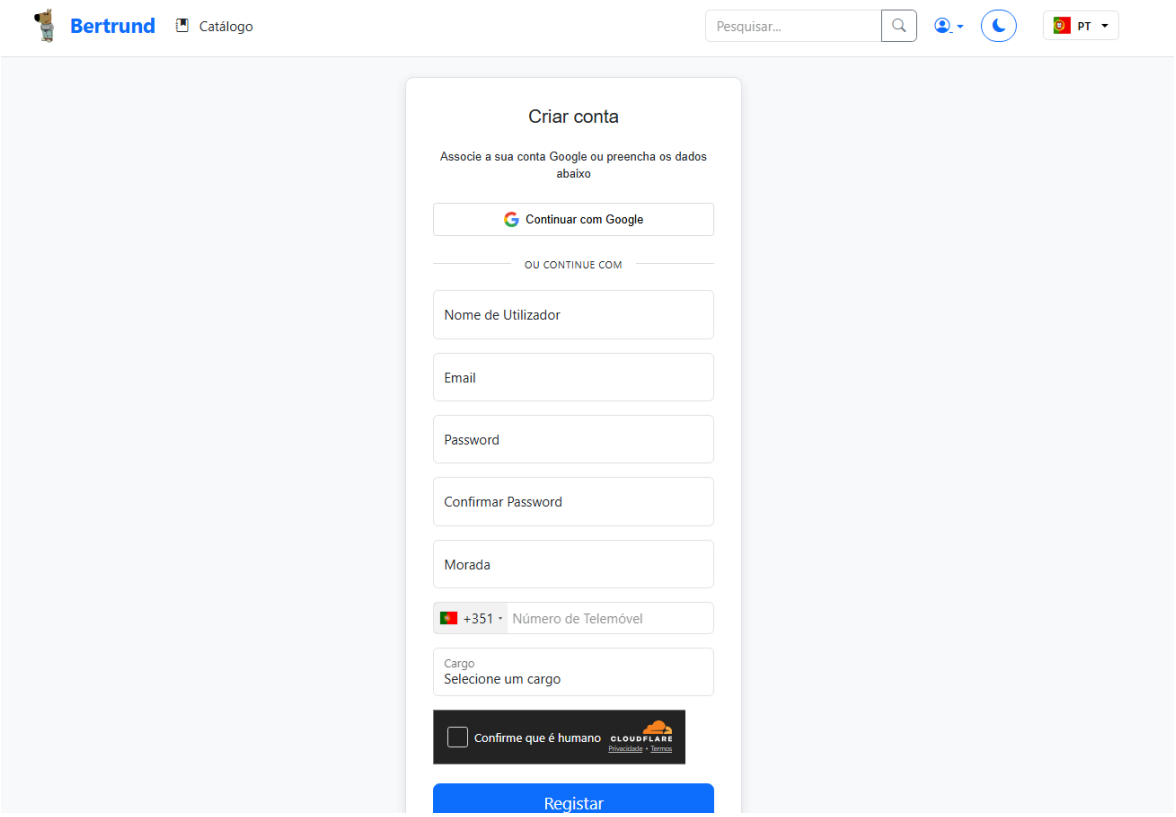


Figura 36 – Página de registo da biblioteca.

Página de registo da biblioteca, onde os utilizadores podem criar uma conta associando o login ao Google ou preenchendo os campos obrigatórios, como nome de utilizador, email, palavra-passe, morada, número de telemóvel e cargo. Inclui verificação CAPTCHA para segurança e um botão para concluir o registo.

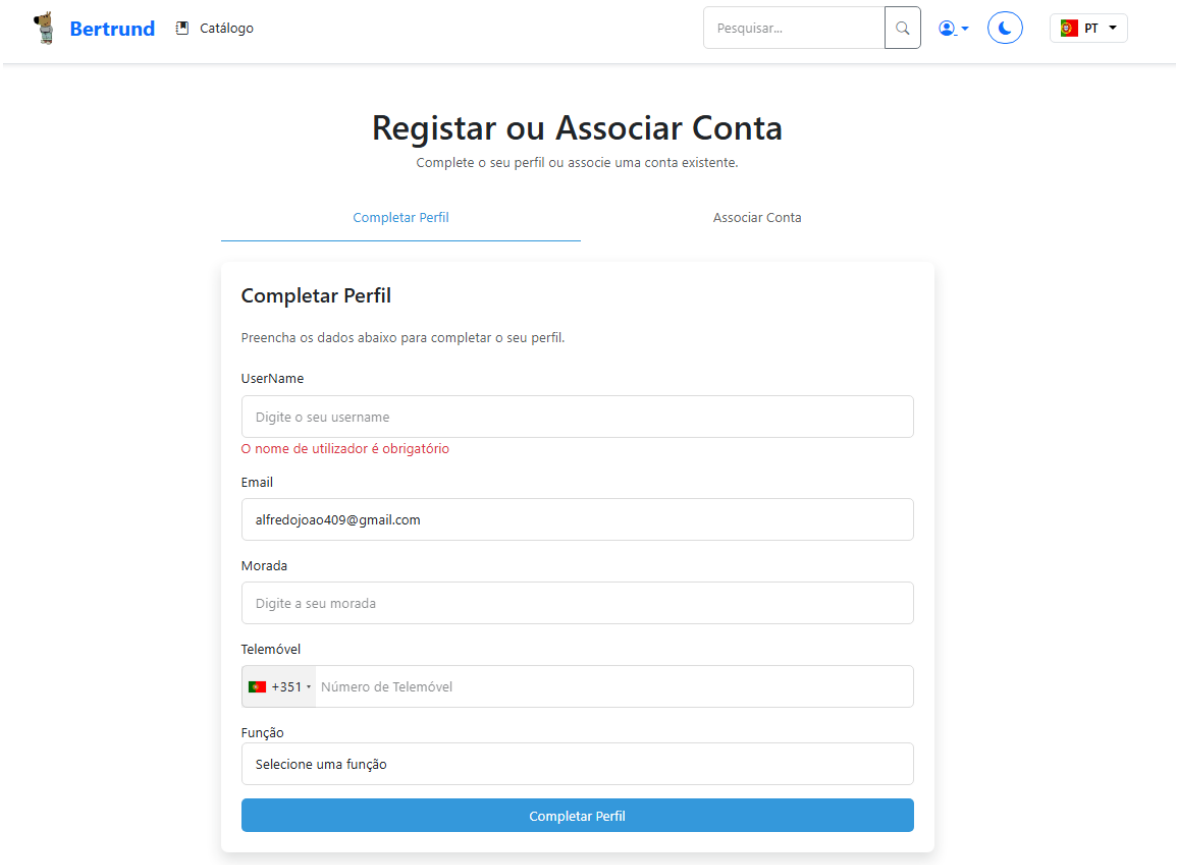


Figura 37 – Página para completar ou associar conta na biblioteca.

Página de "Registar ou Associar Conta" da biblioteca "Bertrund", onde os utilizadores podem completar o seu perfil inserindo informações obrigatórias, como nome de utilizador, email, morada, número de telemóvel e função. A interface inclui validação de campos e um botão para finalizar o preenchimento do perfil.

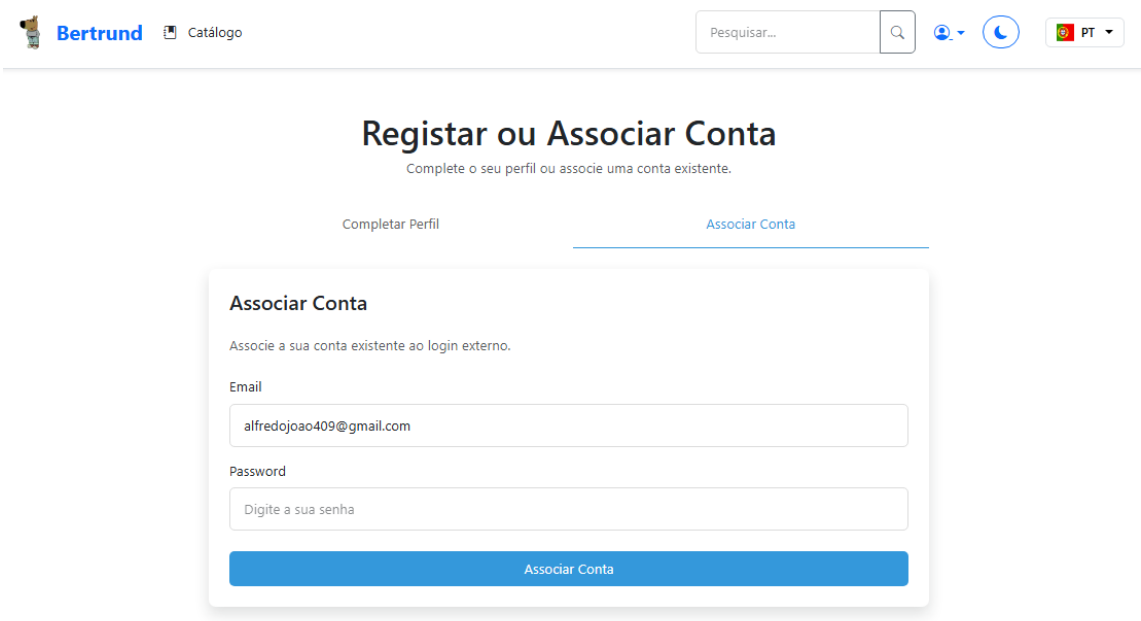


Figura 38 – Página para associar uma conta existente à biblioteca.

Página da funcionalidade "Registar ou Associar Conta" da biblioteca "Bertrund", permitindo aos utilizadores associar uma conta existente ao login externo. Inclui campos para email e palavra-passe, com validação, e um botão para concluir o processo de associação.

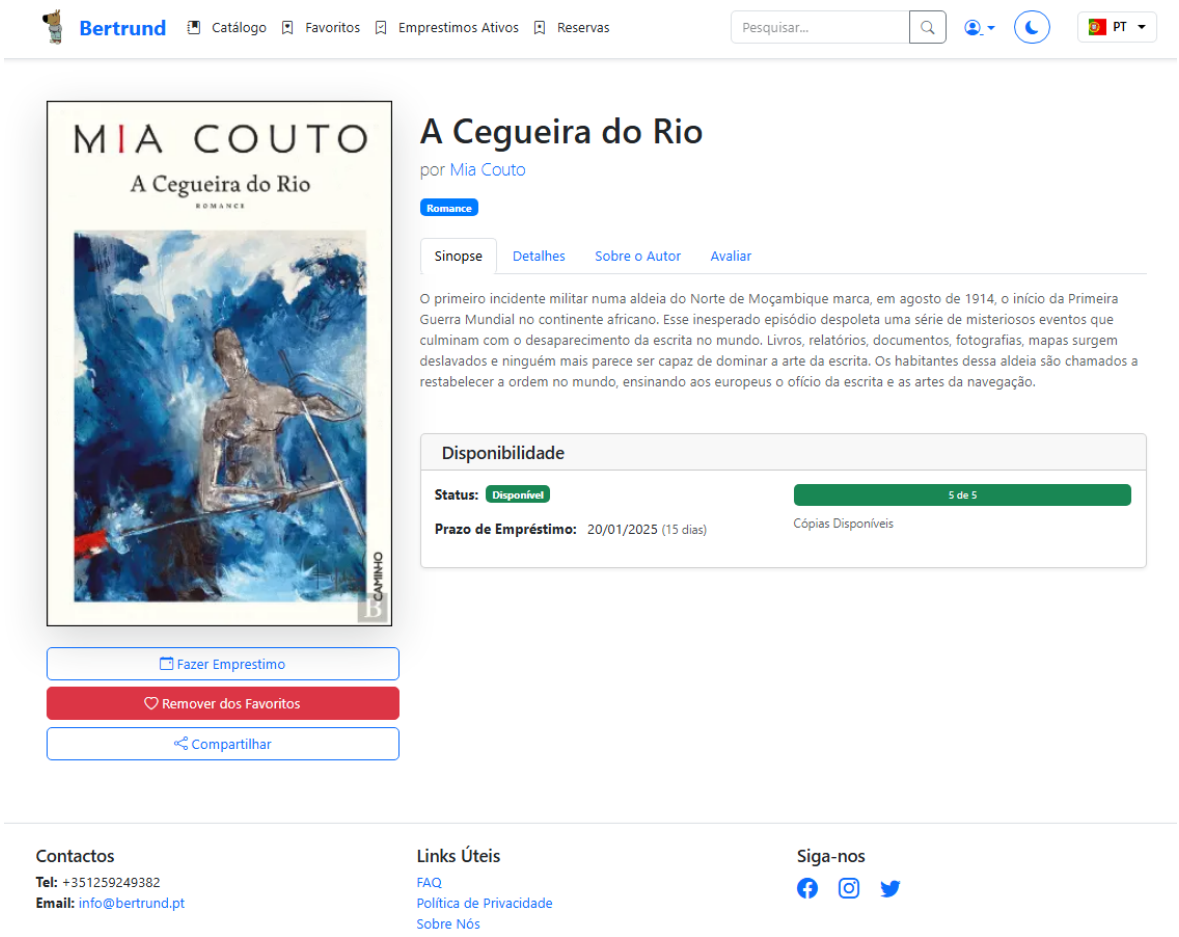


Figura 39 – Página de detalhes de um livro, com opções para empréstimo e informações de disponibilidade.

Página de detalhe do livro *A Cegueira do Rio* de Mia Couto, disponível na biblioteca "Bertrund". Exibe a capa do livro, o género literário (Romance), uma sinopse detalhada e informações sobre o autor. Inclui a secção de disponibilidade, indicando que todas as cópias estão disponíveis, e o prazo de empréstimo de 15 dias. Possui botões para fazer empréstimo, remover dos favoritos e compartilhar, além de contactos e links úteis no rodapé.

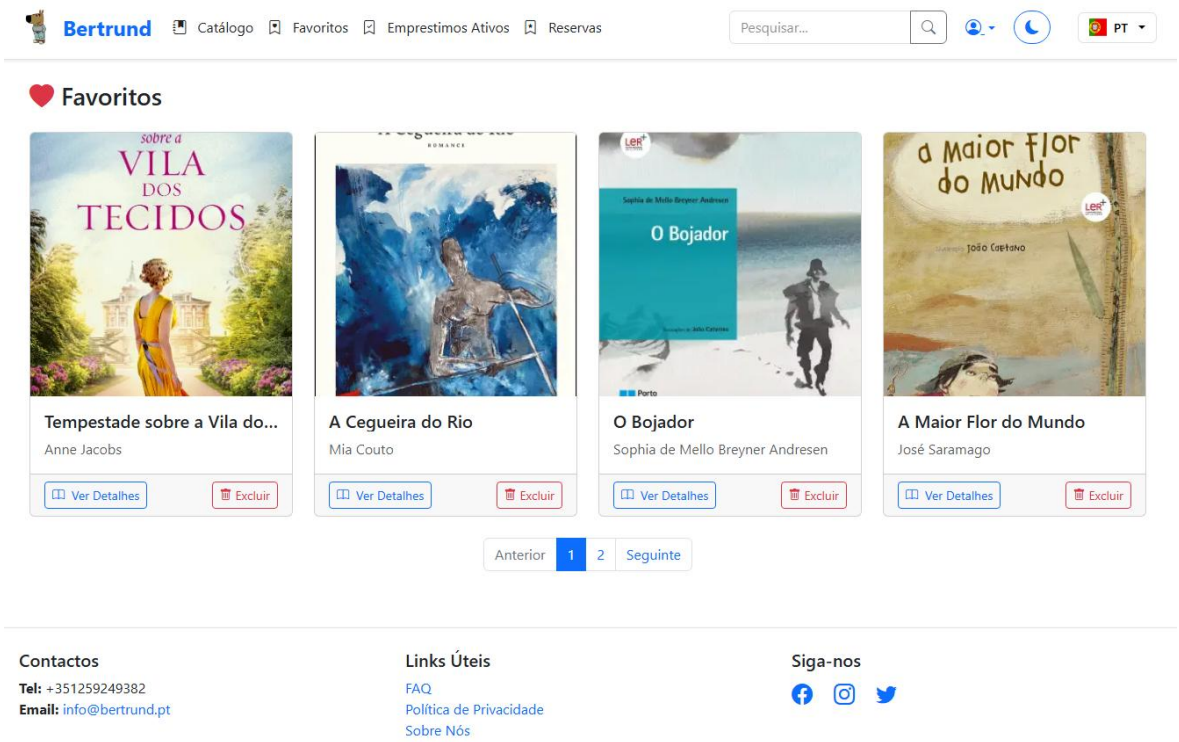


Figura 40 – Secção de Favoritos da biblioteca.

Página da secção "Favoritos" da biblioteca "Bertrund", onde os utilizadores podem visualizar os livros adicionados à sua lista pessoal. Cada livro inclui a capa, título, autor e botões para "Ver Detalhes" ou "Excluir". A navegação é organizada por páginas e o rodapé fornece contactos e links úteis, como FAQs e política de privacidade.

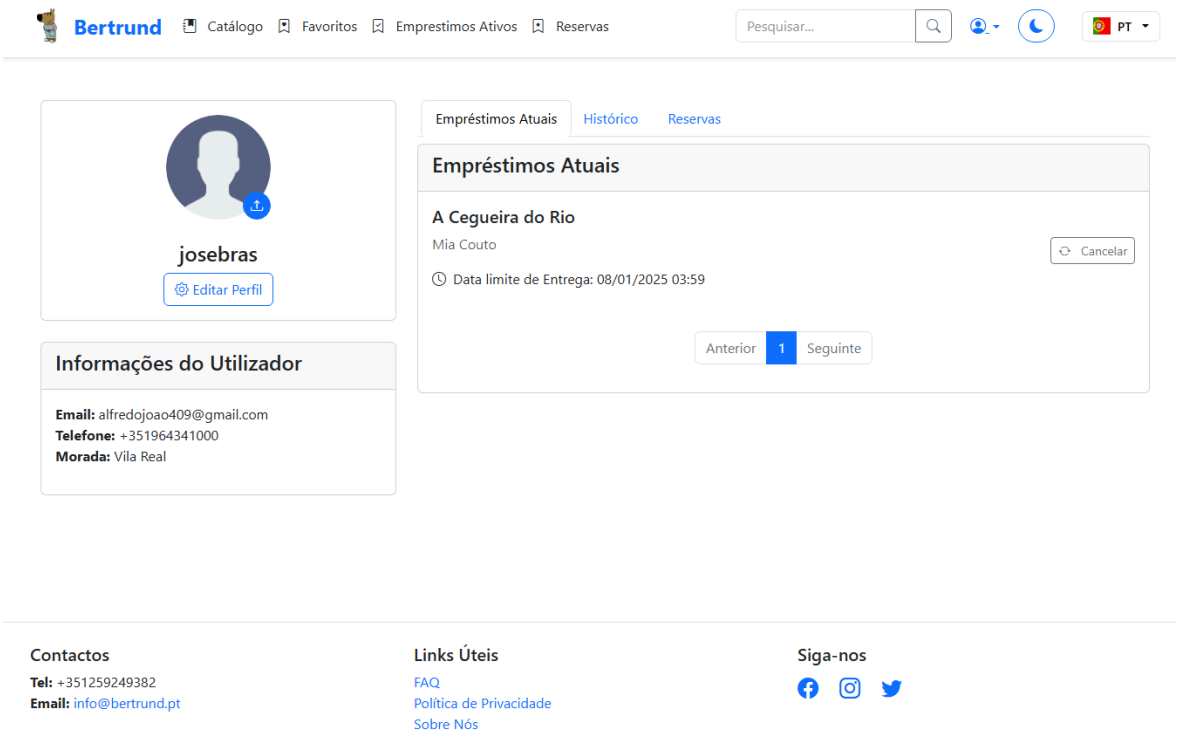


Figura 41 – Empréstimos Atuais do Leitor.

A secção "Empréstimos Atuais" apresenta os livros emprestados, como *A Cegueira do Rio*, com a data-limite de entrega e a opção de cancelar o empréstimo. Inclui navegação para histórico e reservas, bem como contactos e links úteis no rodapé.



Figura 42 – Avaliação do Livro.

Página de detalhes do livro *A Menina do Mar* de Sophia de Mello Breyner Andresen, categorizado como literatura infantil. Exibe a capa, sinopse, informações sobre o autor e uma secção para avaliação, onde os utilizadores podem deixar comentários e atribuir uma classificação por estrelas. Permite ações como fazer empréstimo, adicionar aos favoritos ou compartilhar. A disponibilidade é exibida, indicando 5 cópias disponíveis, com prazo de empréstimo de 15 dias.

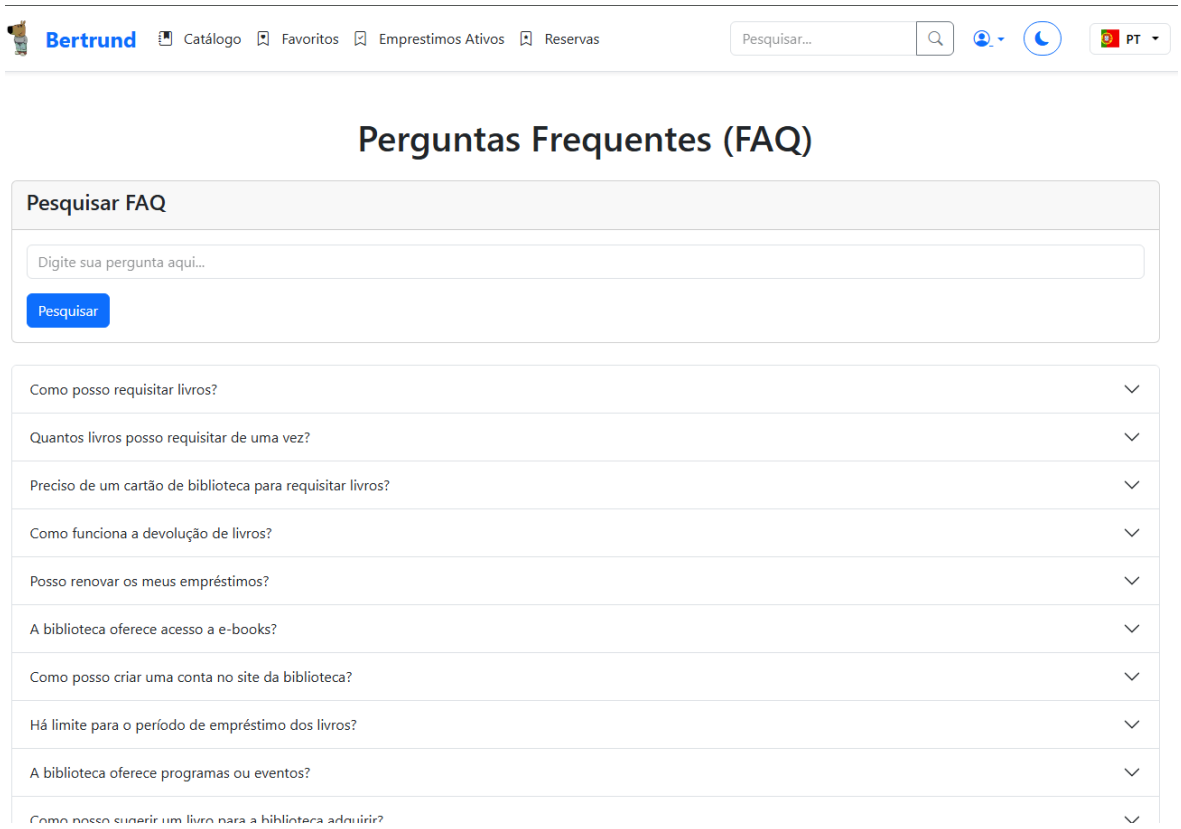


Figura 43 – Página de FAQ da biblioteca.

Página de "Perguntas Frequentes (FAQ)" da biblioteca "Bertrund", com uma barra de pesquisa para facilitar a localização de respostas a questões específicas. Lista tópicos comuns, como empréstimos, devoluções, criação de conta, renovação de empréstimos e acesso a e-books. Cada pergunta pode ser expandida para visualizar a resposta.

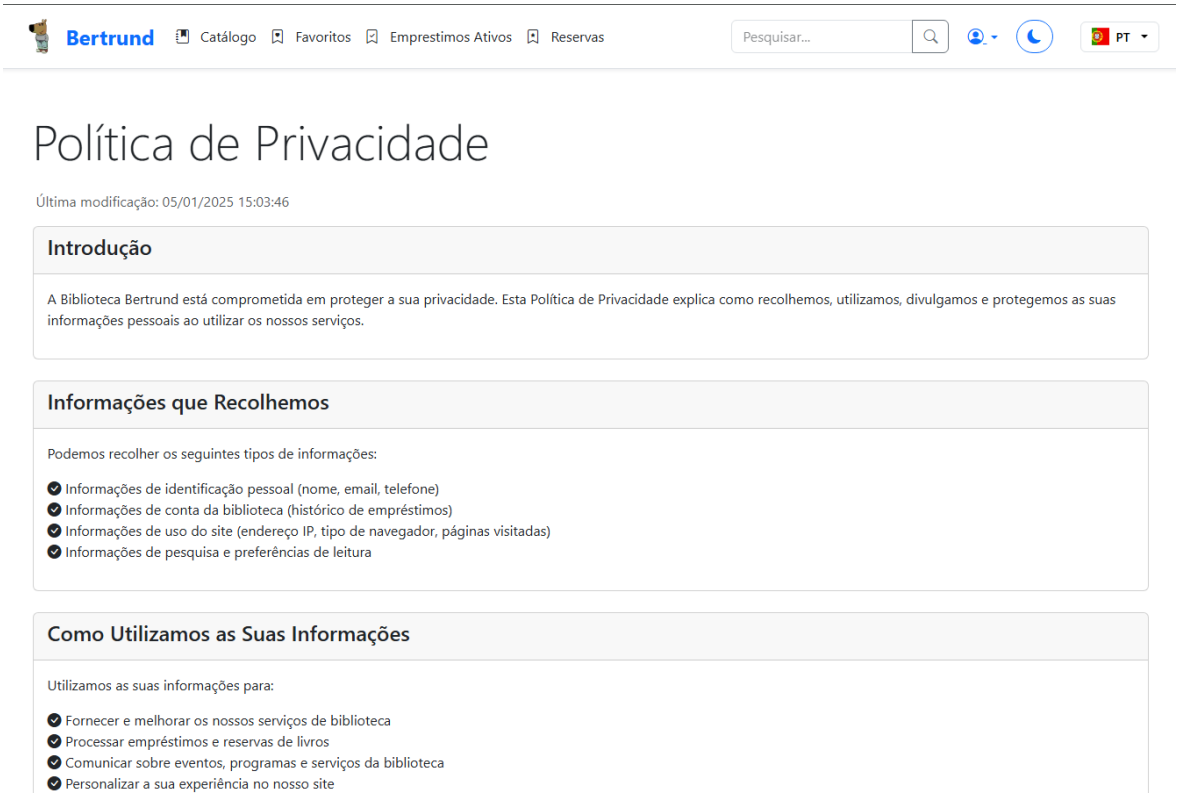


Figura 44 – Página de Política de Privacidade.

Página de "Política de Privacidade" da biblioteca "Bertrund", que explica como são recolhidas, utilizadas, divulgadas e protegidas as informações pessoais dos utilizadores. Divide-se em secções como "Introdução", "Informações que Recolhemos" (incluindo dados de identificação, uso do site e preferências) e "Como Utilizamos as Suas Informações" (com objetivos como melhorar os serviços e personalizar a experiência do utilizador).



Figura 45 – Página 'Sobre a Nossa Biblioteca'.

Página "Sobre a Nossa Biblioteca" da biblioteca "Bertrund", que apresenta a missão e história da instituição. Na secção "Nosso Objetivo", destaca-se a promoção da leitura, o acesso inclusivo a recursos de qualidade e o apoio ao desenvolvimento pessoal. A secção "Nossa História" relata que a biblioteca foi fundada em 1999 e evoluiu para oferecer um ambiente totalmente online, adaptando-se às necessidades digitais dos utilizadores.



Figura 46 – Página informativa sobre o autor, destacando biografia, géneros literários e obras notáveis.

Página dedicada ao autor da biblioteca "Bertrund", apresentando uma biografia detalhada, fotografia e principais informações sobre a sua vida e obra. Inclui atributos como data de nascimento e falecimento, nacionalidade, géneros literários trabalhados e obras notáveis. O formato permite explorar as contribuições literárias e o legado do autor.

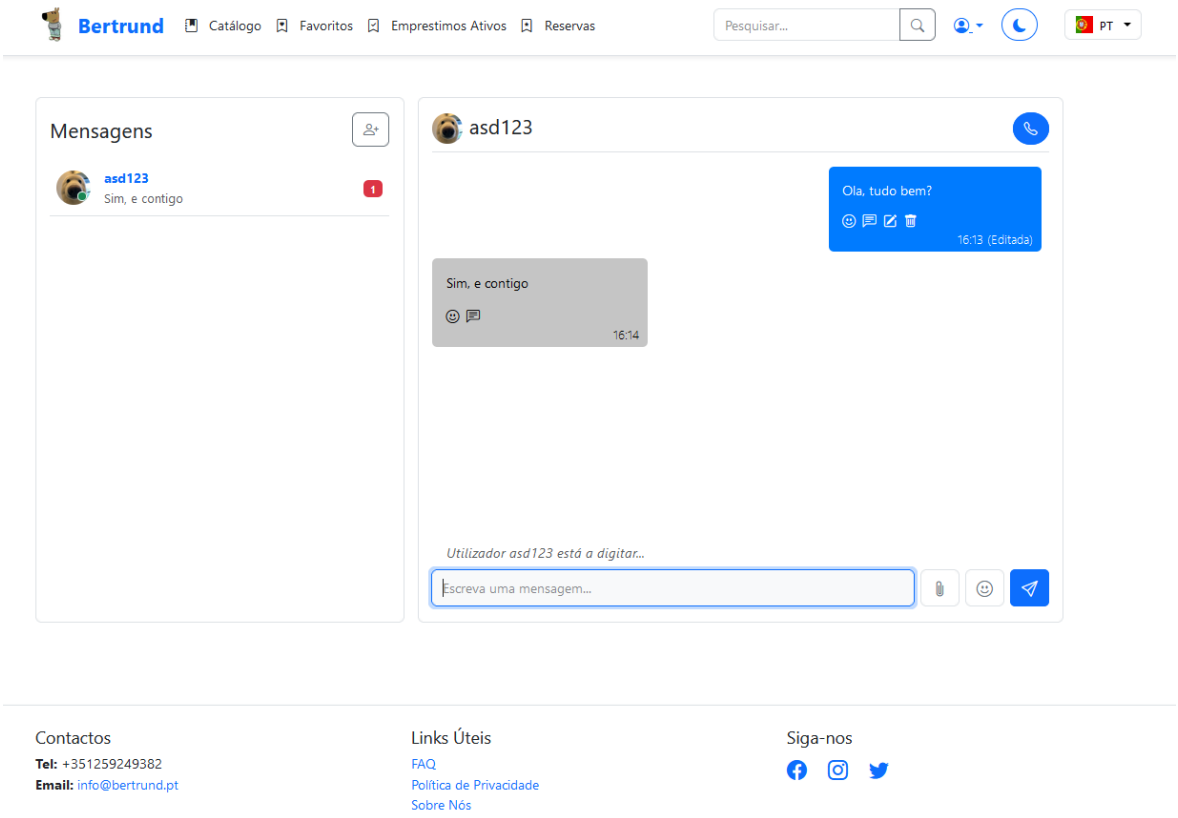


Figura 47 – Funcionalidade de mensagens da biblioteca.

A página apresenta um sistema de mensagens integrado na biblioteca "Bertrund", permitindo aos utilizadores trocarem mensagens em tempo real. A interface inclui uma lista de conversas recentes, visualização de mensagens trocadas, opções de edição e eliminação de mensagens enviadas, bem como indicadores de digitação para acompanhamento em tempo real.

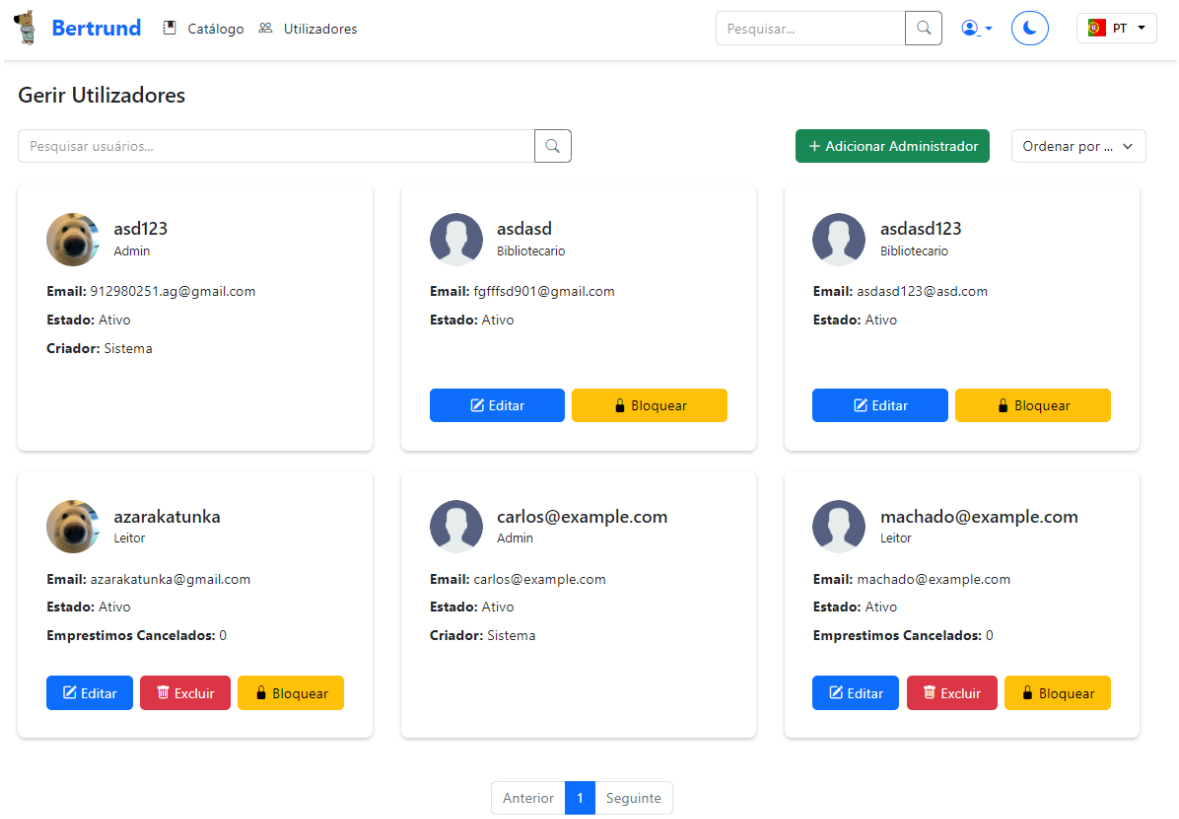


Figura 48 – Funcionalidade de gestão de utilizadores na biblioteca.

A página "Gerir Utilizadores" da biblioteca "Bertrund" permite administrar as contas de utilizadores. Cada perfil apresenta detalhes como nome, email, estado (ativo ou inativo), função (admin, bibliotecário ou leitor) e o criador do perfil. Inclui botões para editar, excluir e bloquear utilizadores, bem como a opção de adicionar novos administradores. A funcionalidade também possui uma barra de pesquisa e um filtro de ordenação para localizar utilizadores rapidamente.

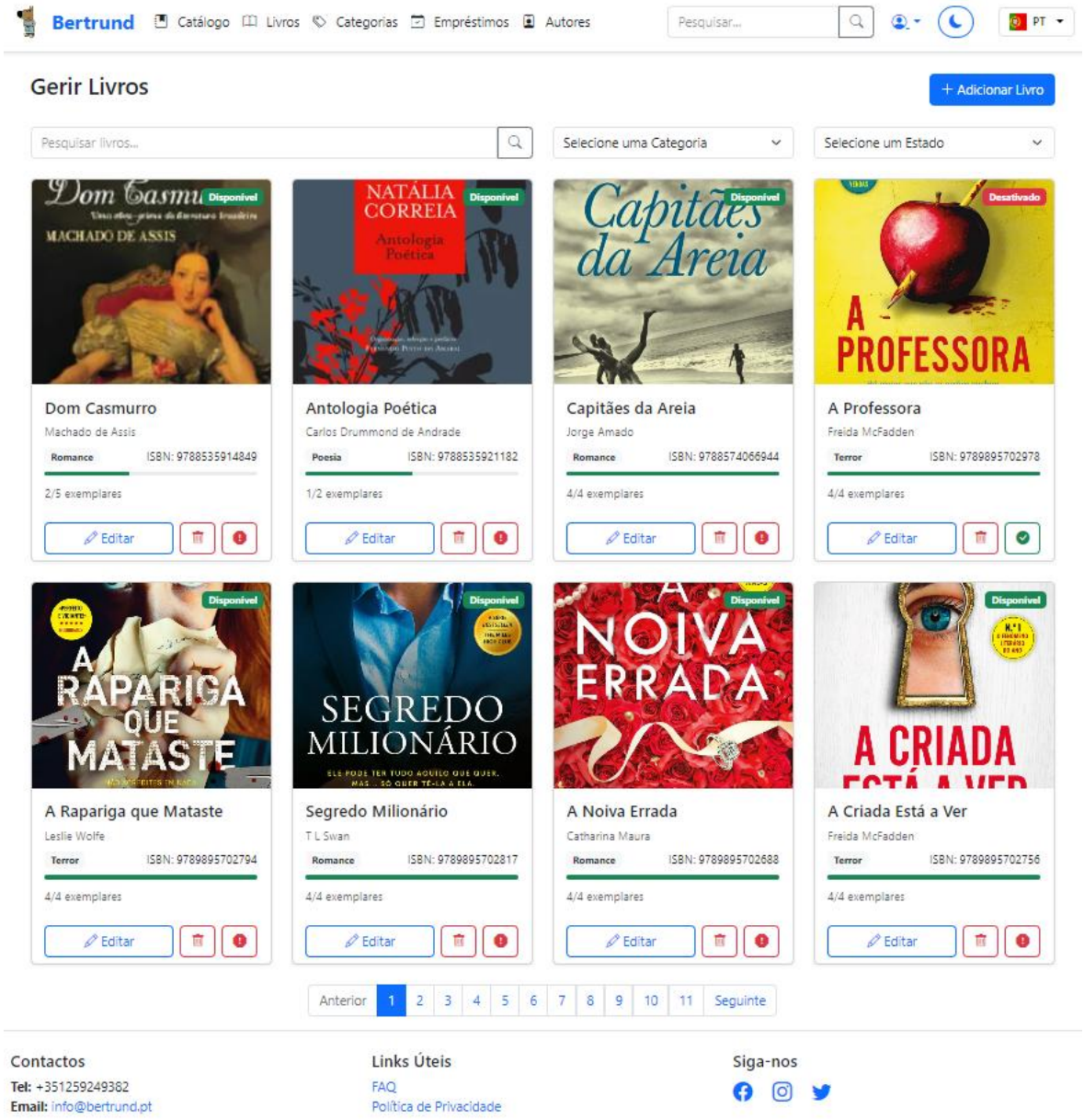


Figura 49 – Funcionalidade de gestão de livros na biblioteca.

A página "Gerir Livros" da biblioteca "Bertrund" permite aos administradores controlar o catálogo de livros. Apresenta informações detalhadas sobre cada livro, incluindo título, autor, ISBN, categoria, estado (disponível ou desativado), e o número de exemplares disponíveis. Oferece funcionalidades para editar, excluir e desativar livros, bem como adicionar novos títulos ao catálogo. Inclui filtros por categoria e estado, além de barra de pesquisa para localizar livros específicos.

4.2 FUNCIONALIDADES IMPORTANTES

Catálogo

```
// GET: Livros/Catalogo
public async Task<IActionResult> Catalogo(int page = 1, string sortOrder = "", List<int>
categorias = null, List<string> estados = null)
{
    ViewData["MessageEmprestimosPerfil"] = TempData["MessageEmprestimosPerfil"];
    var viewModel = new LivroCatalogoViewModel
    {
        PaginaAtual = page,
        TamanhoPagina = 8,
        SortOrder = sortOrder,
        CategoriasSelecionadas = categorias ?? new List<int>(),
        EstadosSelecionados = estados ?? new List<string>()
    };
    var livrosQuery = _context.Livro
        .Include(l => l.Biblioteca)
        .Include(l => l.Categoria)
        .Include(l => l.Autor)
        .AsQueryable();
    livrosQuery = livrosQuery.Where(l => l.Estado != "Desativado");

    // Aplicar ordenação
    switch (sortOrder)
    {
        case "title_desc":
            livrosQuery = livrosQuery.OrderByDescending(l => l.Titulo);
            break;
        case "author":
            livrosQuery = livrosQuery.OrderBy(l => l.Autor.Nome);
            break;
        case "author_desc":
            livrosQuery = livrosQuery.OrderByDescending(l => l.Autor.Nome);
            break;
        default:
            livrosQuery = livrosQuery.OrderBy(l => l.Titulo);
            break;
    }

    // Aplicar filtros
    if (categorias != null && categorias.Any())
    {
        livrosQuery = livrosQuery.Where(l => categorias.Contains(l.CategoriaId));
    }
    if (estados != null && estados.Any())
    {
        livrosQuery = livrosQuery.Where(l => estados.Contains(l.Estado));
    }

    viewModel.TotalLivros = await livrosQuery.CountAsync();
    viewModel.TotalPaginas = (int)Math.Ceiling(viewModel.TotalLivros /
(double)viewModel.TamanhoPagina);
    viewModel.Livros = await livrosQuery
        .Skip((viewModel.PaginaAtual - 1) * viewModel.TamanhoPagina)
        .Take(viewModel.TamanhoPagina)
        .ToListAsync();
    viewModel.Categorias = await _context.Categoria.ToListAsync();

    if (Request.Headers["X-Requested-With"] == "XMLHttpRequest")
    {
        // Requisição AJAX: retorna apenas a view parcial
        return PartialView("BookGrid", viewModel);
    }
    return View(viewModel);
}
```

Figura 50 – Action Catálogo.

O método Catálogo gere a exibição de um catálogo de livros, permitindo paginação, ordenação e filtragem. Inicialmente, cria um LivroCatalogoViewModel para configurar parâmetros como a página atual, tamanho da página e filtros selecionados. A consulta aos livros inclui relacionamentos com Biblioteca, Categoria e Autor, excluindo itens "Desativados". Dependendo do parâmetro sortOrder, os livros são ordenados por título ou autor. Filtros por categorias e estados são aplicados, e os resultados são paginados com Skip e Take. Por fim, retorna uma PartialView se for requisição AJAX ou a view completa.

Pedir Empréstimo

```
[HttpGet]
[Authorize(Roles = "Leitor")]
public async Task<IActionResult> PedirEmprestado(int livroId)
{
    var user = await _userManager.GetUserAsync(User);
    var livro = await _context.Livro.FindAsync(livroId);
    if (user == null || livro == null)
    {
        return NotFound();
    }
    if (livro.NumeroExemplaresDisponiveis <= 0)
    {
        TempData["MessageLivroPage"] = "Error:Não há exemplares disponíveis para empréstimo.";
        return RedirectToAction("Book","Livros", new { id = livroId });
    }
    var emprestimo = new Emprestimo
    {
        LivroId = livroId,
        PerfilId = user.Id,
        DataEmprestimo = DateTime.Now
    };
    livro.NumeroExemplaresDisponiveis--;
    if (livro.NumeroExemplaresDisponiveis == 0)
    {
        livro.Estado = "Indisponivel";
    }
    _context.Emprestimo.Add(emprestimo);
    await _context.SaveChangesAsync();

    ViewData["MessageLivroPage"] = "Success:Pedido de empréstimo efetuado com sucesso!";
    if (Request.Headers["X-Requested-With"] == "XMLHttpRequest")
    {
        var botaoTexto = await GetBotaoTexto(livroId);
        var actionLivro = await GetActionLivro(livroId);
        var modeloFormulario = new FormularioLivroViewModel
        {
            LivroId = livroId,
            BotaoTexto = botaoTexto,
            ActionLivro = actionLivro
        };

        return PartialView("BotaoEmprestimoLivro", modeloFormulario);
    }
    TempData["MessageLivroPage"] = "Success:Pedido de empréstimo efetuado com sucesso!";
    return RedirectToAction("Book","Livros", new { id = livroId });
}
```

Figura 51 – Action PedirEmprestado.

O método PedirEmprestado permite que utilizadores com a role "Leitor" solicitem o empréstimo de um livro. Primeiro, verifica-se se o utilizador e o livro existem; caso contrário, retorna NotFound(). Se não houver exemplares disponíveis, é exibida uma mensagem de erro via TempData e redireciona-se para a página do livro. Caso contrário, cria-se um registo de empréstimo, decrementa-se o número de exemplares disponíveis e, se necessário, atualiza-se o estado do livro para "Indisponível". Os dados são gravados na base de dados, e uma mensagem de sucesso é exibida. Em requisições AJAX, retorna uma PartialView; caso contrário, redireciona-se para a página do livro.

Upload de Imagem de Perfil

```
[HttpPost]
[Authorize]
public async Task<IActionResult> UploadProfileImage(IFormFile profileImage)
{
    if (profileImage != null && profileImage.Length > 0)
    {
        // Lista de extensões permitidas
        var allowedExtensions = new[] { ".jpg", ".jpeg", ".png", ".gif", ".webp" };

        // Verificar a extensão do ficheiro
        var fileExtension = Path.GetExtension(profileImage.FileName).ToLower();
        if (!allowedExtensions.Contains(fileExtension))
        {
            TempData["MessagePerfil"] = "Error:Formato de ficheiro inválido. Apenas imagens
(.jpg, .jpeg, .png, .gif, .webp) são permitidas.";
            return RedirectToAction("Perfil", "Home", new { sectionActivated = "emprestimos-
tab" });
        }

        // Verificar o tipo MIME
        if (!profileImage.ContentType.StartsWith("image/"))
        {
            TempData["MessagePerfil"] = "Error:O ficheiro carregado não é uma imagem válida.";
            return RedirectToAction("Perfil", "Home", new { sectionActivated = "emprestimos-
tab" });
        }

        const long maxFileSize = 10 * 1024 * 1024; // 2 MB
        if (profileImage.Length > maxFileSize)
        {
            TempData["MessagePerfil"] = "Error: O ficheiro é demasiado grande. O tamanho
máximo permitido é de 10 MB.";
            return RedirectToAction("Perfil", "Home", new { sectionActivated = "emprestimos-
tab" });
        }

        var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
        var user = await _userManager.FindByIdAsync(userId);

        // Remove a imagem anterior, se existir
        if (!string.IsNullOrEmpty(user.FotoNome))
        {
            var oldImagePath = Path.Combine(_webHostEnvironment.WebRootPath, "Images/Users",
user.FotoNome);
            if (System.IO.File.Exists(oldImagePath))
            {
                System.IO.File.Delete(oldImagePath);
            }
        }

    // Generate a unique filename
    var fileName = $"{Guid.NewGuid()}_{profileImage.FileName}";
    var filePath = Path.Combine(_webHostEnvironment.WebRootPath, "Images/Users", fileName);

    // Guardar o ficheiro
    try
    {
      using (var fileStream = new FileStream(filePath, FileMode.Create))
      {
        await profileImage.CopyToAsync(fileStream);
      }

      // Atualizar a base de dados
      user.FotoNome = fileName;
      await _userManager.UpdateAsync(user);
    }
    catch (Exception)
    {
      TempData["MessagePerfil"] = "Error: Ocorreu um problema ao guardar a imagem no
servidor.";
      return RedirectToAction("Perfil", "Home", new { sectionActivated = "emprestimos-tab" });
    }

    TempData["MessagePerfil"] = "Success: A imagem de perfil foi atualizada com sucesso!";
    return RedirectToAction("Perfil", "Home", new { sectionActivated = "emprestimos-tab" });
  }

    // Se não foi enviado algum ficheiro, retorna a view Perfil
    return RedirectToAction("Perfil", "Home", new { sectionActivated = "emprestimos-tab" });
}
```

Figura 52 – Action UploadProfileImage.

O método UploadProfileImage permite que utilizadores atualizem a sua imagem de perfil. Verifica se o ficheiro enviado é válido em termos de extensão (permitindo .jpg, .jpeg, .png, .gif, .webp), tipo MIME (apenas imagens) e tamanho máximo de 10 MB. Após validar, remove-se a imagem antiga, se existente, e guarda-se a nova imagem com um nome único no servidor. Caso ocorra um erro no processo de gravação, uma mensagem de erro é exibida; caso contrário, a imagem é atualizada na base de dados. Por fim, redireciona-se o utilizador para a página de perfil, exibindo mensagens de sucesso ou erro via TempData.

Enviar Mensagem

```
[HttpPost]
public async Task<IActionResult> SendMessage(string receiverId, string message, IFormFile?
file, int? replyToMessageId)
{
    if (string.IsNullOrWhiteSpace(message) || string.IsNullOrWhiteSpace(receiverId))
    {
        return BadRequest("Mensagem ou destinatário inválido.");
    }
    //mensagens que eu recebi
    var msgs = await _context.Messages
        .Include(u => u.Sender).Include(u => u.Receiver)
        .Include(u => u.Reactions).Where(m => m.SenderId == receiverId && m.ReceiverId ==
User.FindFirstValue(ClaimTypes.NameIdentifier))
        .OrderByDescending(m => m.Timestamp)
        .ToListAsync();
    //as nao vistas ficam vistas
    var unreadMessages = msgs.Where(m => !m.Seen).ToList();
    foreach (var msg in unreadMessages)
    {
        msg.Seen = true;
    }
    // Guardar as alterações no contexto
    if (unreadMessages.Count > 0)
    {
        await _context.SaveChangesAsync();
    }
    var senderId = User.FindFirstValue(ClaimTypes.NameIdentifier); //quem esta a enviar
    var receiver = await _context.Users.FindAsync(receiverId); //quem vai receber
    if (senderId == null || receiver == null)
    {
        return Unauthorized("Utilizador não autenticado.");
    }
    string? fileName = null;
    // Se houver ficheiro, guarde no servidor
    if (file != null)
    {
        fileName = Guid.NewGuid() + Path.GetExtension(file.FileName);
        var filePath = Path.Combine(Directory.GetCurrentDirectory(), "wwwroot/Files/uploads/",
fileName);
        using (var stream = new FileStream(filePath, FileMode.Create))
        {
            await file.CopyToAsync(stream);
        }
    }
    // Criar a nova mensagem
    var newMessage = new Models.Message
    {
        SenderId = senderId, ReceiverId = receiverId,
        Sender = await _context.Users.FindAsync(senderId),
        Receiver = receiver,Content = message,
        Timestamp = DateTime.Now,FileName = fileName,
        Seen = false,Edited = false,ReplyTo = replyToMessageId
    };
    _context.Messages.Add(newMessage);
    await _context.SaveChangesAsync();
    //encontrar o conteudo da mensagem a dar reply
    var messageToReply = await _context.Messages.Where(m => m.Id ==
replyToMessageId).FirstOrDefaultAsync();
    await _hubContext.Clients.User(receiverId).SendAsync("ReceiveMessage", message,
newMessage.Id,  newMessage.Timestamp.ToString("HH:mm"), newMessage.SenderId,
_chatService.GetIsOnlineUser(newMessage.SenderId), newMessage.Edited, newMessage.ReplyTo,
replyToMessageId == null ? "" : messageToReply.Content.ToString(), newMessage.FileName,
"chatMessage");
    //retornar ou partial view por ajax ou view normal
    if (Request.Headers["X-Requested-With"] == "XMLHttpRequest")
        return await OpenChat(receiverId);

    return RedirectToAction("OpenChat", new { userId = receiverId });
}
```

Figura 53 – Action SendImage.

O método SendMessage permite o envio de mensagens entre utilizadores, incluindo anexos e respostas. Valida se a mensagem e o destinatário são válidos e recupera mensagens anteriores do destinatário, marcando como lidas as não visualizadas. O ficheiro anexado, caso exista, é guardado no servidor com um nome único. Em seguida, cria uma nova mensagem com os detalhes fornecidos (conteúdo, remetente, destinatário, anexo, e mensagem de resposta, se aplicável) e guarda-a na base de dados. Usa SignalR para notificar o destinatário em tempo real, enviando os detalhes da mensagem. Por fim, retorna uma PartialView em requisições AJAX ou redireciona para a função OpenChat.

Notificações

```
function showCallNotification(senderUserId, senderUserName) {
    // Criação do contêiner da notificação
    const notificationDiv = document.createElement('div');
    notificationDiv.className = 'toast show position-fixed top-0 end-0 mt-5 m-3';
    notificationDiv.style = 'min-width: 350px; z-index: 1050;';
    notificationDiv.setAttribute('role', 'alert');
    notificationDiv.setAttribute('aria-live', 'assertive');
    notificationDiv.setAttribute('aria-atomic', 'true');
    notificationDiv.id = `toast-${senderUserId}`;
    // Conteúdo da notificação
    notificationDiv.innerHTML = `
        <div class="toast-body d-flex align-items-center">
            <!-- Avatar -->
            <div class="rounded-circle bg-light d-flex justify-content-center align-items-
center me-3" style="width: 40px; height: 40px;">
                <i class="bi bi-person-circle fs-4" style="font-size: 24px; color: #6c757d;">
</i>
            </div>
            <!-- Informação do Utilizador -->
            <div class="flex-grow-1">
                    <strong>${senderUserName}</strong>
            </div>
            <!-- Botões -->
            <div class="d-flex gap-2">
                    <button class="btn btn-danger btn-sm rounded-circle"
onclick="rejectCall('${senderUserId}')" >
                        <i class="bi bi-x fs-6"></i>
                </button>
                    <button class="btn btn-success btn-sm rounded-circle"
onclick="acceptCall('${senderUserId}', '${senderUserName}')">
                        <i class="bi bi-telephone-inbound-fill"></i>
                </button>
            </div>
        </div>
    `;
    // Adiciona a notificação ao corpo do documento
    document.body.appendChild(notificationDiv);
    // Remove automaticamente após 10 segundos
    setTimeout(() => {
        notificationDiv.remove();
    }, 10000);
}
```

Figura 54 – Função showCallNotification.

```
connection.on("ReceiveCallRequest", (senderUserId, senderUserName) => {
    console.log(`Recebeu uma chamada de ${senderUserId} ${senderUserName}.`);
    //se o user ainda nao esta na lista de users
    if (!document.getElementById("openChat-" + senderUserId)) {
        getUserInfoFromController(senderUserId, 'false') //adicionar o user na lista dos chats
    }
    var isOnCall = document.getElementById("isOnCall-" + senderUserId);
    var lastMessage = document.getElementById("lastMessageUser-" + senderUserId);
    if (isOnCall) {
        isOnCall.classList.remove("d-none");
        isOnCall.classList.add("d-flex"); //mostra que o user esta a ligar-lhe
        lastMessage.classList.add("d-none"); //retira a last message enviado pelo user
    }
    showCallNotification(senderUserId, senderUserName);
});

function acceptCall(senderUserId, senderUserName) {
    console.log(`Chamada de ${senderUserId} aceite.`);
    const openChatHyperlink = document.getElementById(`openChat-${senderUserId}`);
    const chatInitialDiv = document.getElementById("chatArea");
    if (openChatHyperlink) {
        openChatHyperlink.click();
        const intervalId = setInterval(() => {
            const messagesListDiv = document.getElementById("messagesList");

            if (messagesListDiv) {
                clearInterval(intervalId); // Para o intervalo
                startVideoCall(true); //inicia a chamada
                rejectCall(senderUserId);
            }
        }, 100); // Verifica a cada 100ms
    }
    else {
        //faz fetch ao controller para obter as infos sobre o senderUserId, pois nao sei a
image e o isOnline e adiciona o user e aceita a chamada
        getUserInfoFromController(senderUserId);

    }
}
```

Figura 55 – Função acceptCall.

O código gere notificações e interações relacionadas a chamadas recebidas em tempo real, utilizando SignalR e manipulação do DOM. O evento ReceiveCallRequest é ativado quando se recebe uma chamada, verificando se o utilizador remetente já está na lista de chats. Caso não esteja, adiciona-o através de getUserInfoFromController. O estado visual do DOM é alterado para mostrar que o utilizador está a ligar, e uma notificação é exibida através da função showCallNotification, que cria dinamicamente um componente com opções para aceitar ou rejeitar a chamada. Esta notificação desaparece automaticamente após 10 segundos. Ao aceitar a chamada com acceptCall, o chat do utilizador é aberto, e, após verificar se a área de mensagens está carregada, inicia-se a chamada de vídeo. Se o remetente não estiver previamente na lista de chats, as informações são obtidas do servidor antes de aceitar a chamada.

5 CONSIDERAÇÕES FINAIS

A fase 1 estabeleceu os alicerces do projeto ao clarificar necessidades e funcionalidades do sistema. Foram definidas as principais entidades e relações através de diagramas E-R e especificados os casos de uso para estruturar as interações entre utilizadores e sistema. Com requisitos e modelagem inicial concluídos, o projeto segue com direção clara, reduzindo riscos e preparando-se para o design e implementação nas próximas fases.

A fase 2 aprofundou o desenvolvimento do projeto ao detalhar a execução do trabalho e realizar o mapeamento entre o modelo conceptual de dados e o modelo relacional. Através da implementação do modelo físico da base de dados em SQL, foram estruturados os fundamentos técnicos para o armazenamento de dados. Além disso, especificaram-se as interfaces de utilizador com *mockups*, delineando a interação esperada com o sistema. Com estas etapas concluídas, o projeto avança com uma base sólida para as próximas fases de refinamento e testes.

Na fase 3, o foco principal esteve na finalização e integração do sistema. Esta etapa incluiu a elaboração de um relatório detalhado que documenta a execução do trabalho, descrevendo cada componente desenvolvido e os processos implementados. Paralelamente, garantiu-se a integridade da base de dados, assegurando que todos os dados são manipulados de forma consistente e segura. Além disso, foi implementada a lógica funcional do sistema, permitindo estabelecer a ligação entre as interfaces de utilizador e a base de dados. Com esta etapa, o sistema ficou totalmente funcional, alinhando a interface intuitiva com os fundamentos técnicos robustos, e preparando-o para validação final e testes.